IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 14 April 2026, accepted 4 May 2026, date of publication 11 May 2026, date of current version 14 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3692203

RESEARCH ARTICLE

Time-Scale Reconfiguration for Ransomware Mitigation by Delaying Encryption Progress

KOSUKE HIGUCHI AND RYOTARO KOBAYASHI, (Member, IEEE)
Kogakuin University, Shinjuku-ku, Tokyo 163-8677, Japan

Corresponding author: Kosuke Higuchi (ed25001@g.kogakuin.jp)

This work was supported in part by Japan Society for the Promotion of Science (JSPS) KAKENHI under Grant 23K11108.

ABSTRACT Recent crypto ransomware can amplify damage within minutes by combining highly parallel encryption with fast file replacement. Because detection latency cannot be reduced to zero even with accurate detectors, effective defense requires damage mitigation that secures recoverability ahead of encryption progress. This paper extends the Real-time Open-File Backup System (ROFBS), which creates backups upon file open events, and addresses cases where ROFBS fails under high-throughput or bursty encryption. The failure is modeled as a Fundamental Time-Scale Mismatch, where the backup completion time exceeds the effective time window before plaintext becomes irreversibly unavailable. To restore temporal slack without relying solely on faster backups, the paper proposes Time-Scale Reconfiguration (TSR), which intervenes in encryption execution timing and compute-resource allocation. TSR is implemented as ROFBS-TSR using extended Berkeley Packet Filter (eBPF) to observe protected-file open events and to apply modular delay controls. Evaluated controls include process suspension, CPU-share throttling via cgroup v2, scheduler-layer control via sched_ext (SCX with scx_flatcg), and CPU-parallelism throttling via cpuset. Experiments on multiple Linux ransomware samples (Sodinokibi, IceFire, HelloKitty, AvosLocker, Conti, Monti, REvil, and LockBit) quantify backup completion, encrypted-target coverage, and backup precision using Cov_B, EwB, and Pre_B together with counts (N_E, N_S, N_B, and N_{EB}). Results show that suspension effectiveness is sensitive to the application-window design, scheduler-layer choice can substantially change outcomes under identical weights, and parallelism throttling is beneficial only for specific families. Overall, TSR is most effective when early intervention is achieved and when the dominant bottleneck matches the selected control lever.

INDEX TERMS Ransomware, damage mitigation, real-time open-file backup, time-scale reconfiguration, temporal slack, eBPF, cgroup, sched_ext.

I. INTRODUCTION

Ransomware can be broadly classified into two categories: crypto ransomware, which encrypts victims' files and data to demand ransomware, and locker ransomware, which locks victims' computers and prevents normal operations [1]. In recent years, crypto ransomware has become a dominant threat, and the number of incidents has continued to increase. According to a CyberEdge report, 72.7% of organizations were affected by ransomware attacks in 2023, which is the highest proportion since 2018 [2]. Moreover, attacks have become increasingly sophisticated and diversified, and Allianz forecasts that global annual damages could reach approximately 265 billion USD by 2031 [3].

Existing countermeasures against ransomware can be broadly categorized into detection and damage mitigation (defense), and recent research has particularly emphasized improving detection accuracy. Traditional signature-based methods identify malware using static features such as filenames and hashes, but they are weak against variants and previously unseen samples [4]. Motivated by this limitation, many studies have investigated behavior-based detection using machine learning. For example, Amjad and Abdulmohsen showed that machine learning models can learn the

The associate editor coordinating the review of this manuscript and approving it for publication was Cong Pu.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

behavioral patterns of ransomware and detect anomalies [5], and Beaman et al. report that process-behavior-based classifiers can achieve high detection performance with sufficient training data [6].

However, as pointed out by Kok et al., research on defense (damage-mitigation) models remains relatively limited compared to detection [7]. Even with highly accurate detectors, the detection latency cannot be reduced to zero, leaving a time window in which encryption can progress and cause irreversible damage. This challenge is amplified by modern ransomware, which often employs highly parallel encryption (multi-threading) and fast file replacement, enabling the rapid destruction of endpoint user data. As a result, relying only on scheduled backups or post-incident recovery is frequently insufficient to preserve the most recent state of data. Therefore, rather than merely starting backups after detecting encryption activity, it is necessary to control the time scale of encryption progress itself and delay encryption until backups complete.

In our prior work, we proposed the Real-time Open-File Backup System (ROFBS), a defense model that creates backups in real time using file open events as triggers, and demonstrated that it can substantially mitigate encryption damage under certain conditions [8]. Nevertheless, when ransomware performs high-throughput or bursty encryption, ROFBS may fail to keep up, resulting in degraded recoverability. To formalize this competition, let E_{time} denote the effective per-file time window until plaintext is irreversibly lost, and let B_{time} denote the per-file time required to complete a valid backup (including overheads such as event handling and metadata operations). Real-time backup can follow encryption only if

$$B_{\text{time}} \leq E_{\text{time}} \tag{1}$$

holds. Conversely, persistent violation ($B_{\text{time}} > E_{\text{time}}$) represents a fundamental conflict caused by a mismatch between the time scales of attack (encryption progress) and defense (backup completion), rather than an implementation bug. In this paper, we refer to this condition as a Fundamental Time-Scale Mismatch.

This study builds on ROFBS and investigates a framework that does not rely solely on accelerating backups. Specifically, we intervene in the execution timing of encryption and the allocation of compute resources to artificially extend the effective time window E_{time} and delay encryption progress until backups complete. We call this approach Time-Scale Reconfiguration (TSR). The goal of TSR is to create temporal slack by controlling the speed of encryption progress, thereby stabilizing damage mitigation even under conditions dominated by a Fundamental Time-Scale Mismatch. We refer to the extended scheme that integrates TSR-based delay control into ROFBS as ROFBS-TSR. ROFBS-TSR intervenes in the time scale of encryption progress upon observing file open events, aiming to restore the ability of backups to keep pace by delaying encryption until backups complete. In addition, we adopt an architecture that separates the detection process from the backup process to minimize implementation dependencies when comparing delay-control mechanisms.

In ROFBS TSR, we use Extended Berkeley Packet Filter (eBPF) to detect file open events with low overhead and implement delay control as a replaceable module that is applied upon accesses to protected files. From the two perspectives of intervening in execution timing and intervening in compute resource allocation, we apply and compare the following control mechanisms. These mechanisms include pause based delay using SIGSTOP and SIGCONT, CPU share control using cgroup v2 (cpu.weight), scheduler layer control using sched_ext (SCX) with scx_flatcg combined with cgroup weight control, and CPU parallelism restriction using cpuset. This design enables us to separately evaluate whether backups can be created at all and whether backups can keep pace with encrypted targets. It also allows us to clarify how behavioral differences among ransomware, such as highly parallel encryption, file replacement, and note generation, affect the effectiveness of delay based defenses.

The experiments are organized from four perspectives. First, we compare fixed duration and backup completion application windows for suspension based control and evaluate how window design affects the ability to keep pace. Second, we compare the conventional scheduler (CFS) and an extensible scheduler (SCX) under a fixed cpu.weight to isolate the impact of the scheduler layer on encryption delay and tracking. Third, we compare fixed duration and backup completion application windows for CPU parallelism restriction (cpuset) to assess effective conditions and limitations against highly parallel encryption. Fourth, we evaluate the impact of delay control on benign workloads using GPG and file copying to a LUKS mounted encrypted volume in order to assess practical deployability. In addition, we conduct a preliminary sensitivity analysis on representative samples to examine the dependence on fixed parameter settings. We evaluate these settings using multiple Linux ransomware samples, namely Sodinokibi, IceFire, HelloKitty, AvosLocker, Conti, Monti, REvil, and LockBit.

To facilitate interpretation of results, we adopt metrics that separate feasibility, coverage, and precision. Specifically, we introduce Cov_B to represent the feasibility of backup creation, EwB to represent coverage of encrypted targets by backups, and Pre_B to represent the precision of generated backups with respect to encrypted targets. By analyzing these ratios together with the corresponding absolute counts N_E, N_S, N_B, and N_{EB}, we can quantitatively distinguish situations in which backups are created but do not keep pace with encrypted targets.

Note that this study does not propose a hardware assisted protection mechanism. Its scope is limited to an OS level software based design that realizes delay control using techniques available on Linux, including file open observation via eBPF, process suspension, CPU share control via cgroup, scheduler

layer control via sched_ext, and CPU parallelism restriction via cpuset.

The main contributions of this paper are summarized as follows.

1) We systematize a TSR based damage mitigation framework that manipulates the time scale of encryption progress starting from file open observation and implement it through multiple control mechanisms, including process suspension, cgroup weight control under CFS and SCX, and CPU parallelism restriction.
2) We present an evaluation methodology that separates feasibility and coverage using Cov_B and EwB, and further quantifies backup precision using Pre_B, enabling bottleneck analysis and method comparison even under harsh conditions.
3) Using multiple ransomware samples, we experimentally characterize how application windows, the scheduler layer, and CPU parallelism restriction influence tracking and overall effectiveness.
4) We further complement the core evaluation with a preliminary sensitivity analysis and a benign workload evaluation, thereby clarifying both parameter dependence and practical deployability.

Section I reviews related work on ransomware detection and damage mitigation. Section III summarizes the kernel level observation technique used in this study (eBPF), defines Fundamental Time Scale Mismatch as a time scale mismatch between encryption and backup, and illustrates how it manifests in ROFBS behavior. Section IV describes Time Scale Reconfiguration (TSR) and the design and implementation of ROFBS TSR, which increases temporal slack by intervening in encryption execution timing and compute resource allocation. Section V describes the experimental setup, workloads, and metrics. Section VI reports the results. Section VII discusses the findings and limitations. Section VIII concludes the paper and outlines future work.

II. RELATED WORK

This section reviews prior studies on ransomware defense from both the hardware side and the software side. The former is discussed in Subsection II-1, and the latter is discussed in Subsection II-2. These studies are surveyed in order to position the present work within the broader landscape of ransomware countermeasure research. Note that the proposed ROFBS TSR is an OS level software based design and does not require dedicated hardware, SSD internal logic, or device level modification.

1) HARDWARE-AND DEVICE-ASSISTED APPROACHES

Several studies have explored ransomware defense by offloading protection logic to storage devices or by leveraging device-level mechanisms. Gujar et al. proposed software that detects newly added files in real time and immediately backs them up to a dedicated folder inside an SSD. Because the protection software resides on the SSD and is separated from the host machine, the approach can provide stronger data protection than software running on a potentially compromised host. At the same time, the authors noted challenges such as optimizations for specific SSD models and real-time processing overheads [9].

Reidys et al. proposed a ransomware-aware SSD (RSSD) that implements hardware-assisted logging inside the SSD to address emerging attacks that target SSD behavior, such as forcing garbage collection, intentionally slowing encryption pace, and abusing TRIM commands [10]. Their evaluation indicated that RSSD can defend against new and future ransomware attacks with little performance overhead.

Baek et al. proposed SSD-Insider++, a storage-level defense system designed to prevent file damage under ransomware attacks. By separating protection from the host machine, the system aims to be more robust than software-based approaches that are vulnerable to evasion. They reported that SSD-Insider++ detects ransomware with high accuracy in most cases and enables instantaneous recovery with 0% data loss [11].

Mir et al. proposed Cache Assisted Ransomware Detection and Recovery (CARDR), which leverages SSD DRAM caches to counter ransomware at the SSD level [12]. In their evaluation, CARDR achieved faster detection than SSD-Insider++ (approximately 11 seconds vs. 14 seconds) and reduced recovery queue size by about 42% compared to a baseline.

Fujinoki and Manukonda proposed PDPZR, a backup mechanism that creates an immediate backup on each data update and removes obsolete backups. Their simulation results suggest that PDPZR is a promising solution for mitigating damage caused by zero-day ransomware, while highlighting the need for more diverse experiments and careful configuration and tuning of automatic protection policies [13].

Zhu et al. propose LAST, an intrusion-detection-aware versioning system for flash-based SSDs [14]. LAST reportedly achieves only a 1.5% latency overhead compared to conventional SSDs while retaining data history for up to 126.4 days (and 52.6 days on average). The authors also report that this retention period exceeds that of existing versioning approaches by at least 61.4% and by as much as 165.9%. Zhu et al. also propose SrFTL, a mechanism that monitors I/O activity at the SSD/FTL layer, transfers OS-level file information to a TEE to determine whether behavior is ransomware-like, and, upon detection, delays the deletion of affected data by suspending garbage collection and related processes [15]. According to their experiments, SrFTL achieves zero false positives and enables rapid data recovery with an average recovery time of 9.3 s against evaluated ransomware samples.

Min et al. propose AMOEBA, a device-level backup solution that does not require additional storage space for backups [16]. They implement AMOEBA using a Microsoft SSD simulator and further prototype it on the

OpenSSD platform. Their evaluation indicates negligible performance overhead while achieving high ransomware detection accuracy.

Friday and Bou-Harb propose a method for shifting the entire Long Short-Term Memory (LSTM)-based classification process to Computational Storage Drives (CSDs) [17]. They argue that this approach reduces CPU load in data centers under rapidly increasing data volumes and that directly offloading prompt ransomware defense to CSDs can effectively protect data from encryption.

2) SOFTWARE-BASED APPROACHES

Hernández et al. developed R-Locker, a defense system that leverages decoy files to prevent ransomware-induced file damage by monitoring file-access behavior [18]. By placing decoy files and observing their access patterns, the system can stop malicious activity before critical user files are encrypted.

Medhat et al. proposed a hybrid approach for detecting packed ransomware samples by scanning both process memory dumps and dropped executable files. By strengthening a YARA-rule framework and describing common ransomware characteristics, their method reportedly achieves detection on approximately 98% of dump files.

Zhuravchak et al. proposed a honeypot-based method to mitigate file-system damage. Their approach monitors accesses to honeypot files and identifies the responsible ransomware process accordingly [19].

Lee et al. focused on file extensions hard-coded in ransomware binaries and proposed a protection method that randomizes the extensions of important files. They reported that this approach successfully prevented encryption for 141 out of 143 ransomware samples [20].

A common limitation of these approaches is that they rely on assumptions such as decoy files being accessed early or a specific extension-selection strategy (e.g., whitelisting) being used by the adversary. Therefore, they may be ineffective against ransomware that encrypts files in a randomized order or employs blacklist-based selection.

Recent studies have also emphasized pre-encryption detection and proactive protection mechanisms. Kok et al. proposed a machine-learning-based Pre-Encryption Detection Algorithmto detect crypto ransomware before encryption starts. Their evaluation reported a low false positive rate of 1.56% and a high AUC of 99.3%, concluding that it is a strong tool for detecting and blocking diverse crypto ransomware attacks [21].

Song et al. proposed an Android-oriented mechanism that detects anomalous behaviors using processor utilization, memory usage, and I/O rate. Upon detection, the system suspends the ransomware process, prompts the user for confirmation, and then removes the program to protect user data, without maintaining explicit ransomware signatures [22].

Cusack et al. proposed a stream-processing approach built on a programmable forwarding engine (PFE) and used a random-forest classifier to identify malicious network activities. Their experiments showed that flow-based fingerprinting can achieve sufficient accuracy for pre-encryption ransomware detection [23].

Beyond detection, several works have studied backup-centric mitigation.

Oujezsky et al. proposed the Intelligent MalwareDefense System (IMDS) to reduce infection risk in backup data using AI and hash functions. IMDS verifies data integrity before backup and blocks backup when anomalies are detected, arguing that such a design can provide stronger protection than conventional passive backup schemes [24].

Ahmed et al. proposed Profiling kernel-Level Events to detect Ransomware (Peeler), which detects ransomware by profiling system-level activities of files and processes via kernel-level event monitoring [25]. They evaluated Peeler on a large dataset including 67 ransomware families and reported an F1 score of 99.5%.

Ganfure et al. proposed a systematic framework that generates deceptive files using machine learning and detects/contains ransomware through interactions with those decoys [26]. They reported that RTrap can detect ransomware with an average loss of 18 files per 10,311 legitimate user files.

Anand et al. proposed Linux Anti-Ransomware Monitor (LARM), a lightweight real-time detection tool tailored for Linux systems on the x86_64 architecture [27]. LARM combines eBPF-based real-time monitoring with non-parametric clustering on affinity propagation and heuristics on encryption-order behaviors, dynamically selecting trap files for monitoring. Their evaluation reported an average detection latency of 1,240 ms and a file loss rate of 0.46%, demonstrating LARM's effectiveness for early detection and mitigation on Linux.

Wang et al. propose Cancal, a system that selectively filters suspicious processes at the monitoring layer and detects ransomware through in-depth analysis [28]. Based on a five-month evaluation, they report a true positive rate of 99.65%, near-zero false positives, inference within 30 ms, and real-time response within at most 3 s.

von der Assen et al. propose GuardFS, a file-system-based approach for investigating the integration of ransomware detection and mitigation [29]. Their evaluation in a reactive environment shows that, although data leakage cannot be completely prevented, it can be substantially reduced.

III. BACKGROUND AND PROBLEM SETTING

A. EXTENDED BERKELEY PACKET FILTER

eBPF is a mechanism that executes restricted bytecode programs in the Linux kernel and allows observation and control logic to be dynamically attached to various in-kernel hook points [30]. By using eBPF, custom processing can be introduced for kernel events without modifying the kernel source code or building and loading kernel modules, unlike conventional kernel extension techniques. At load time, the eBPF verifier checks safety properties of the program

(e.g., preventing invalid memory accesses and enforcing termination), which reduces the risk of destabilizing the system due to faulty extensions. As a result, eBPF has been widely used for application tracing, performance analysis, network observability, and security monitoring.

Historically, eBPF programs may depend on internal kernel data structures (e.g., structure layouts), which can lead to portability issues across kernel versions. Recent mechanisms such as Compile Once–Run Everywhere (CO-RE) together with BPF Type Format (BTF) address this issue by improving portability across kernel versions.

BPF Compiler Collection (BCC) is an eBPF development and operational framework from the IOVisor project, providing libraries and a suite of tools for tracing and monitoring [31]. In BCC, the in-kernel eBPF program is written in C, compiled into eBPF bytecode via LLVM/clang, loaded into the kernel, and attached to selected hook points. User-space logic for program orchestration and for processing and storing collected data can be written in Python or Lua, enabling a clear separation between in-kernel execution and user-space data handling. In addition, BCC ships with practical, task-oriented tools, making it suitable for rapid instrumentation and measurement.

B. FUNDAMENTAL TIME-SCALE MISMATCH

This paper interprets the phenomenon that conventional ROFBS fails to keep pace with backup creation under certain conditions as a mismatch between the time scales of encryption (attack) and backup (defense). When this mismatch persists, backup delay becomes self-amplifying and the backlog grows over time. We refer to this regime as a Fundamental Time-Scale Mismatch. Fig. 1 shows a typical case in which backups fail to keep up under continuous encryption.

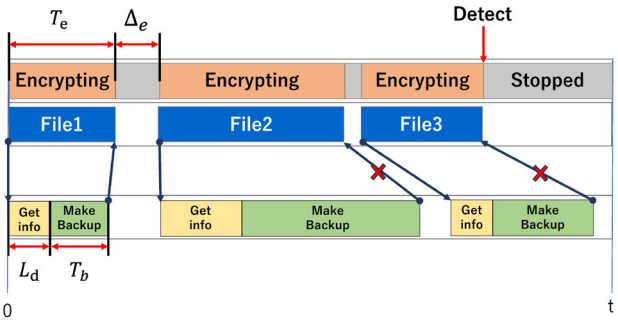


FIGURE 1. Continuous encryption scenario where backups fail to keep pace.

To connect the abstract time windows E_{time} and B_{time} to quantities observed in our experiments, we further decompose them as follows. Let T_e denote the per-file encryption time, Δ_e denote the inter-encryption interval, L_d denote the initial latency from open detection to backup start, and T_b denote the per-file backup processing time. For ROFBS to track the encryption stream, the backup-side time spent per encryption event, $L_d + T_b$, must not exceed the attacker-side time budget, $T_e + \Delta_e$. The primary cause of tracking failure can be summarized as follows:

$$T_b + L_d \;>\; T_e + \Delta_e. \tag{2}$$

To simplify the discussion, we introduce temporal slack S as the time difference between encryption and backup:

$$S := (T_e + \Delta_e) - (L_d + T_b). \tag{3}$$

When $S > 0$, sufficient slack exists per encryption event, and the backup process can keep pace with the encryption event stream. In contrast, when $S \leq 0$, Eq. (2) holds, incomplete backups accumulate, and the backlog grows. In this paper, we regard the regime in which $S \leq 0$ broadly holds and the time-scale mismatch dominates as a Fundamental Time-Scale Mismatch.

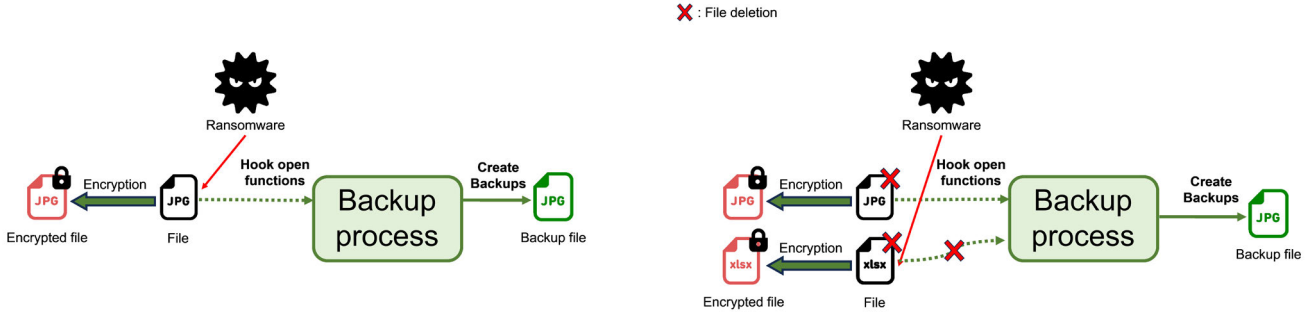
C. ROFBS UNDER FUNDAMENTAL TIME-SCALE MISMATCH: AN ILLUSTRATIVE EXAMPLE

Real-time Open-File Backup System (ROFBS) is a defense model that creates backups in real time using file open events as triggers [8]. Its primary goal is to secure a backup copy of a target file before encryption or replacement occurs, thereby improving recoverability. Fig. 2 illustrates the behavior of conventional ROFBS under continuous encryption. In the figure, red crosses indicate that the original files were deleted during the encryption process, whereas green backup files indicate successful backup creation. As shown in Fig. 2a, conventional ROFBS performs backups sequentially on a per-file basis. However, because ransomware encryption often proceeds with high continuity and short inter-event intervals, even if the first backup completes in time, subsequent files may be encrypted and deleted before the corresponding backups complete, as shown in Fig. 2b. Operationally, failures manifest as cases where encryption or deletion preempts backup initiation, or where the original file is deleted while the backup is still being written.

Based on the temporal slack S defined in Eq. (3), a backup may succeed for the first target file where S is non-negative, while tracking can collapse when S becomes negative for subsequent files. In Fig. 1, encryption or deletion can interrupt before backup initiation, and the original file can be removed mid-backup. In the regime where $S \leq 0$, incomplete backups systematically increase for $k \geq 2$, leading to missing backups for encrypted targets.

Importantly, this behavior is not limited to workloads in which the inter-encryption interval Δ_e is always small. In workloads that mix small files with very large files, a transient spike in T_b can push S negative, and the resulting backlog can propagate to subsequent files. In other words, even if the average Δ_e is moderate, a single peak load event (large T_b) can trigger a cascade of failures, sharply degrading tracking.

While ROFBS can mitigate damage under favorable conditions via sequential backups, it lacks a mechanism to intervene in the time scale of encryption progress to restore



(a) Conventional ROFBS: sequential per-file backup.

(b) Backup lag after the first file due to continuous encryption.

FIGURE 2. **Conventional ROFBS behavior under continuous encryption.**

S to positive once it has fallen into the $S \leq 0$ regime. Therefore, once ROFBS enters this regime, recovering tracking performance becomes difficult unless the encryption throughput decreases.

IV. TIME-SCALE RECONFIGURATION

A. DEFINITION OF TIME-SCALE RECONFIGURATION

In the previous section, we showed that when the temporal slack S becomes negative (Eq. (3)) due to a time-scale mismatch between encryption and backup, incomplete backups cascade and the ability to keep pace degrades systematically. This regime, in which the mismatch dominates, is referred to as a Fundamental Time-Scale Mismatch. A key limitation of conventional ROFBS is that it cannot intervene in the time scale of encryption progress itself. Once the system falls into the $S \leq 0$ regime, there is no mechanism to restore S to a positive value, which constitutes the gap addressed in this work.

To bridge this gap, this study adopts a design principle that intentionally delays encryption progress so that the condition under which backups can "make it in time" is restored. Specifically, upon file access (open), the defense artificially stretches the effective encryption time and/or the inter-encryption interval, thereby recovering the temporal slack S. This framework is termed Time-Scale Reconfiguration (TSR).

The goal of TSR is not to compete solely by accelerating backup processing, but to manipulate the time scale of encryption progress itself so that the system transitions from the $S \leq 0$ regime to a regime that satisfies $S \geq 0$.

B. TWO METHODS FOR TIME-SCALE RECONFIGURATION

TSR can be conceptually organized into two methods of intervention against encryption progress.

The first method intervenes in execution timing. It suppresses the execution of the encryption process for a period of time and explicitly inserts waiting time into the encryption event sequence. Process suspension and resumption (e.g., temporarily stopping and later continuing execution) are representative examples. Fig. 3a illustrates delaying encryption progress triggered by the first open event on a protected file. Fig. 3b illustrates that, by introducing such delay, backup processing is more likely to advance ahead of encryption, making it more likely that backups for subsequent files are completed before encryption and deletion occur. Fig. 3 provides an overview of ROFBS-TSR, where encryption progress is intentionally delayed to create temporal slack for real-time backups.

The second method intervenes in compute resources. It throttles the CPU time and/or the degree of parallelism allocated to the encryption process, reducing encryption throughput and effectively stretching encryption time and/or the inter-encryption interval. In this paper, CPU-share control via cgroup v2 using cpu.weight, scheduler-layer control via sched_ext (SCX) using scx_flatcg combined with cgroup weight control, and CPU-parallelism restriction via cpuset are treated as concrete instances of this method.

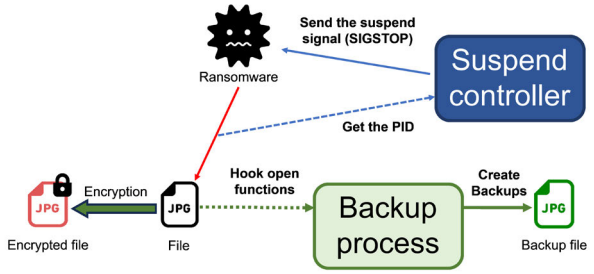
In the remainder of this paper, these interventions are collectively referred to as delay control. The period during which delay control is active is referred to as the delay window. For suspension-based control, the delay window appears as an explicit pause interval, whereas for CPU-share control or parallelism throttling, encryption continues to progress but at a reduced speed. Fig. 4 conceptually illustrates how the delay window expands the inter-encryption interval and secures additional time margin for backup processing.

To generalize how the tracking condition changes with delay control, the analysis is expressed in terms of temporal slack S. Assume that delay control increases the total time budget for an encryption event by $\tau > 0$, and that applying and releasing the control introduces a constant overhead $\delta \geq 0$. Then, the tracking condition can be rewritten as follows:

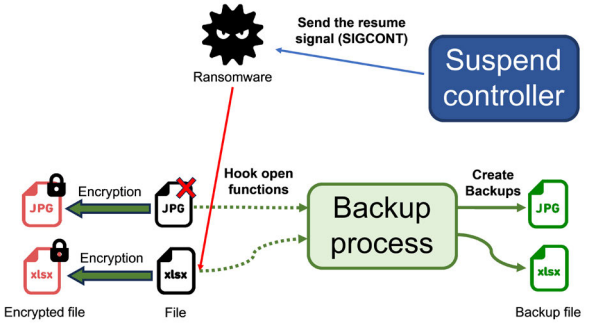
$$T_b + L_d + \delta \leq T_e + \Delta_e + \tau. \tag{4}$$

Using the slack definition in Eq. (3), the new slack after applying TSR, denoted by S_{tsr}, is given by

$$S_{\mathrm{tsr}} := (T_e + \Delta_e + \tau) - (L_d + T_b + \delta)$$



(a) Triggering time-scale reconfiguration on the first protected-file open to delay encryption progress.



(b) Completing sequential backups within the reconstructed time window before encryption advances.

FIGURE 3. ROFBS-TSR overview: delaying encryption progress to create temporal slack for real-time backups.

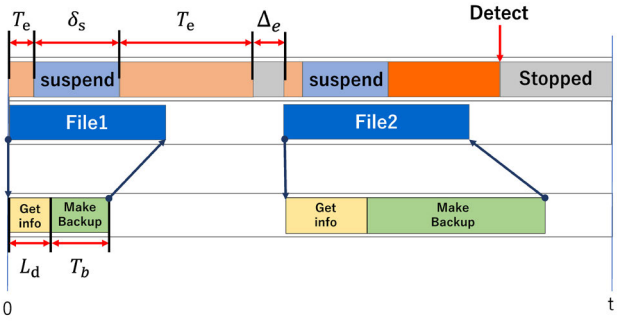


FIGURE 4. Expanding the inter-encryption interval with intentional delay control.

$$= \underbrace{\big((T_e + \Delta_e) - (L_d + T_b)\big)}_{S} + \tau - \delta$$
$$= S + \tau - \delta. \tag{5}$$

Equivalently to Eq. (4), sequential tracking holds when $S_{\text{tsr}} \geq 0$, meaning that backups can "make it in time" with respect to encryption.

Importantly, even when the original slack S is negative, tracking can be recovered if the additional time provided by delay control, $\tau - \delta$, is sufficiently large such that

$$S_{\text{tsr}} = S + \tau - \delta \ > \ 0.$$

Thus, TSR can restore the ability to keep pace by reconfiguring the time scale even under Fundamental Time-Scale Mismatch conditions where $S \leq 0$ is dominant.

C. APPLICATION WINDOW

The effect and side effects of delay control depend on when it is activated and how long it is maintained. This paper defines two types of application windows and compares them experimentally.

In the fixed-duration window, delay control is activated upon the first open event (or detector alert) and released after a predetermined duration W. This design is straightforward and provides an explicit upper bound on the control duration. However, it may release the control before backups have completed, or conversely maintain unnecessary delay even after backups have finished.

In the backup-completion window, delay control is activated upon an open event and maintained until the system is judged to have returned to a trackable state. Concretely, the release condition is tied to the completion of the relevant backup processing and the draining of the backup backlog. This design can secure the amount of time needed for tracking recovery in a workload-adaptive manner. At the same time, it requires state management to decide when to release the control, and the control duration may become long depending on the situation.

These window definitions are orthogonal to the two intervention methods described above (execution timing and compute resources) and can be combined independently. In the following experiments, the impact of the choice of delay mechanism and application window on Cov_B and EwB is evaluated.

V. EXPERIMENTS

A. SETTINGS AND WORKLOADS

Table 1 summarizes the software-based evaluation environment used in this study. The experiments were conducted on an AlmaLinux 10.1 guest VM running on VirtualBox over a Windows 11 host. The protected directory was placed on XFS, and file-open events were observed via xfs_file_open. These specifications are reported to facilitate reproducibility and do not imply that the proposed method is hardware-assisted or requires a virtual-machine-based deployment.

TABLE 1. Experimental environment (host and guest VM).

	Host machine	Guest VM (VirtualBox)
CPU	Intel Core i7-12650H	Virtual processor, 4 cores
Memory	32 GB (DDR4 2400 MT/s)	16 GB
Storage	512 GB (M.2 NVMe)	100 GB
OS	Windows 11	AlmaLinux 10.1

To evaluate how delay control for encryption progress affects the tracking capability of ROFBS, experiments were conducted using eight ransomware samples: Sodinokibi, IceFire, HelloKitty, AvosLocker, Conti, Monti, REvil, and LockBit. In this paper, the sample labels follow the names assigned at the time of sample collection and signature identification. The samples used in this study were collected from the Internet between May and October 2023 and in March 2026. Table 2 lists the MD5 hashes of the samples used in the experiments.

For all experimental conditions, five independent runs were performed, and the reported values in the result tables correspond to the mean and standard deviation across runs. Accordingly, although N_E, N_S, N_B, and N_{EB} are count-based quantities defined on unique base names, they are reported in the revised manuscript as averages over five runs.

TABLE 2. Ransomware samples used in the experiments.

Ransomware	MD5
IceFire	01de715b0f9e3725ef453d31acaaf598
HelloKitty	f597d74b84929ded948fc42c6f6210e5
AvosLocker	d3cafcd46dea26c39dec17ca132e5138
Conti	cfb6d21ffe7c4279f761f2351c0810ee
Sodinokibi	c83df66c46bcbc05cd987661882ff061
Monti	486bd1fe562ce0c339a6c0ec8df68284
REvil	96a157e4c0bef22e0cea1299f88d4745
LockBit	ca93d47bcc55e2e1bd4a679afc8e2e25

The protected directory is a set of user files placed under /home/user/victim/. Backup processing is triggered only when accesses to paths under /home/user/victim/ are observed.

Unlike the prior ROFBS implementation, this study does not execute detection and ROFBS backup creation within a single process. Instead, the detection process and the backup process are separated and executed as distinct processes [32]. This design aims to reduce interference between model inference and backup generation, and to minimize implementation dependencies when comparing different delay-control mechanisms. In the following experiments, this separated architecture is used consistently across all conditions, including the baseline, and only the control methods listed in Table 3 are varied.

The trigger is file open detection using eBPF. The file path is reconstructed from the open event and matched against the protected directory to determine the backup target. For all control methods, the PID and PPID of the execution entity that issued the open are identified, and delay control is applied to both.

In an open-triggered backup scheme, auxiliary files such as ransom notes may be observed earlier than user data. In such cases, backup processing may be attracted to these auxiliary files, delaying backups for user data. Although ransom notes are typically low priority for recovery, they can be created and placed before encryption begins, which can exacerbate backlog growth. For example, AvosLocker has been reported to place ransom notes in each directory before starting encryption [33]. In addition, prior studies report that ransom-note filenames and contents frequently include terms such as ‘‘decrypt’’ and ‘‘file’’, and that filenames resembling ‘‘ReadMe’’ or including decryption-instruction keywords are commonly used [34].

As a practical mitigation, this study filters backup targets at the selection stage. Specifically, if the filename matches the ransom-note naming patterns reported in [34], the file is excluded from backup and the backup routine for that open event is terminated immediately. This policy suppresses backlog inflation caused by preferentially backing up ransom notes and prioritizes tracking for user data.

In this study, the additional delay associated with file-open observation is treated as part of L_d. However, L_d does not refer only to the delay of the eBPF hook itself; rather, it is interpreted as a non-zero latency component that depends on both the implementation and the execution environment, including hook execution, event collection, and user-space dispatch. Jia et al. reported that conventional system-call filtering can incur context-switch overhead due to user-space agents, whereas eBPF-based filtering can reduce such overhead [35]. Vieira et al. also describe eBPF/XDP as a foundation for high-performance in-kernel packet processing, supporting the view that eBPF is widely used as a low-overhead in-kernel processing substrate [36]. Accordingly, this paper does not assume that the observation latency introduced by eBPF is zero; instead, L_d is positioned as a practical delay term that may vary with the implementation and the execution environment.

B. EXPERIMENTAL FACTORS

The compared conditions consist of a baseline without delay control (ROFBS) and multiple variants with delay control. Across the main ransomware experiments, the dataset, protected paths, open detection method, and backup generation logic are fixed. Only the delay control mechanism and the application window are varied.

The main experimental design is organized along three comparison axes. First, for suspension based control, fixed duration and backup completion windows are compared. Second, for CPU share control, the scheduler layer is compared between CFS and SCX while using the same cpu.weight=1 setting. Third, for CPU parallelism restriction, fixed duration and backup completion windows are compared. In addition to these three comparison axes, the study also includes a preliminary sensitivity analysis on representative samples to examine dependence on fixed parameter settings, and a benign workload evaluation to assess operational side effects and practical deployability.

Table 3 summarizes the main delay control conditions used in the ransomware experiments. The backup completion window applies delay control from the moment a backup target is determined after open detection until backup generation for that target completes. The fixed duration window

starts delay control upon open detection and releases it after a predetermined duration. When accesses to protected files are observed continuously, these applications are repeated accordingly.

Among these conditions, scheduling based control is evaluated only under the backup completion window. This is because the primary objective of CPU share control is to suppress encryption throughput until backups complete and thereby secure temporal slack for backup creation. In the SCX condition, scx_flatcg is started in advance and sched_ext is kept enabled throughout the experiments so that the observed differences can be primarily attributed to the scheduler layer.

Evaluation is performed using Cov_B, EwB, and Pre_B. In particular, this study focuses on cases where backups are created but do not keep pace with encrypted targets (high Cov_B but low EwB), as well as cases where the number of backups increases but many do not correspond to encrypted targets (low Pre_B). These metrics enable discussion of how CPU allocation differences between CFS and SCX affect time scale reconfiguration. The preliminary sensitivity analysis and the benign workload evaluation are presented separately because they serve complementary purposes: the former clarifies parameter dependence, whereas the latter clarifies deployability under legitimate workloads.

C. DETECTION MODELS

This study adopts the detector used in the prior ROFBS work and applies the same configuration to both ROFBS and ROFBS-TSR. Because the primary goal is to compare delay-control mechanisms, the training procedure and hyperparameters of the detectors are kept identical to the prior work, and no retuning is performed in this study.

File-operation features used as detector inputs are collected via eBPF. Specifically, read/write-related information is obtained using kprobes, while other events such as renaming are obtained using kretprobes. Table 4 lists the kernel functions hooked in this study and their corresponding behaviors.

Only the information required for detection is selected and stored in log form. Specifically, the logged fields include timestamps, per-process cumulative read and write volumes, the number of deletes, the number of renames, and the number of file-open events. While kprobes/kretprobes can capture a wide range of information such as function arguments and return values, collecting features that are not used for detection can unnecessarily increase system resource consumption. Therefore, the collected fields are restricted to what is sufficient for the intended detection.

After a sufficient number of logs has accumulated, the system transitions to the decision phase. In the decision phase, logs are not necessarily ordered by time, and multiple logs for the same PID can occur at nearly the same time, which makes it difficult to interpret ordering relationships without preprocessing. Accordingly, logs are first sorted by timestamp to establish a time-ordered sequence. Next, logs from the same PID occurring within a short time window are aggregated as a single event by summing read/write counts to reduce excessive fragmentation. Then, read and write volumes from adjacent logs are appended to each entry, and the resulting sequence is parsed into a consistent representation suitable as input to the detection model. The overall flow of the decision phase is shown in Algorithm 1.

Feature logs for training and testing were collected in an environment separate from the experimental environment used in this study. Logs obtained from executions of both ransomware and benign software were organized into pseudo-temporal sequences following Algorithm 1. To evaluate generalization to unseen ransomware families, Monti, REvil, and LockBit were treated as unseen families, whereas the remaining ransomware families were included in the training data. Benign logs were collected from common benign activities, including file archiving with tar and zip, as well as routine shell commands such as ls and cd.

Some ransomware families spawn multiple child processes during execution, which makes activity tracking inaccurate if cumulative statistics are computed using only PID. To address this issue, this study associates logs using PPID at collection time so that activities by child processes can be tracked as part of the same lineage.

Algorithm 1 Algorithm for the Judgment Phase (log Parsing and inference)

```
function data_parser(target_logs)
    Load the model.
    Sort the target_logs based on time.
    if there is a log from the same pid within a certain time window then
        Consider them as part of the same event and accumulate read/write counts.
    else
        Add log information (e.g., time, PID, cumulative reads/writes) to parsed_data.
    end if
    For each log, also add read/write information from the data before and after to parsed_data.
    Make predictions for parsed_data and store the results.
    for each result in pred do
        Save the PID and the decision result of each log.
    end for
    if judged as malignant then
        Trigger the response for the process and its child processes.
    end if
end function
```

To run the detector, four parameters are configured: (i) the total number of malicious decisions required to trigger a stop action, (ii) the decision threshold for judging an output as malicious, (iii) the number of logs required for a decision, and (iv) the time width used to treat multiple logs as the same event. This study adopts the same parameter values

TABLE 3. Experimental conditions and control methods.

Condition	Description
ROFBS (baseline)	After open detection, only backup creation is performed; no delay control is applied.
Suspend (fixed)	After open detection, the PID/PPID is suspended for a fixed duration (3 s) and then resumed.
Suspend (backup-completion window)	After open detection, the PID/PPID is suspended until backup generation completes, and then resumed.
Scheduling (CFS + cgroup)	After open detection, the PID/PPID is moved to a dedicated cgroup and cpu.weight=1 is applied. The control is applied only within the backup-completion window and then reverted [37], [38].
Scheduling (SCX + scx_flatcg + cgroup)	Before the experiment starts, scx_flatcg is launched and sched_ext remains enabled throughout the experiment. After open detection, the PID/PPID is moved to a dedicated cgroup and cpu.weight=1 is applied. The control is applied only within the backup-completion window and then reverted [37], [39].
CPU parallelism (cpuset, backup-completion window)	After open detection, cpuset.cpus=1 is applied to the PID/PPID and maintained until backup generation completes, and then reverted [37].
CPU parallelism (cpuset, fixed)	After open detection, cpuset.cpus=1 is applied to the PID/PPID, maintained for a fixed duration (3 s), and then reverted [37].

TABLE 4. Kernel functions hooked by eBPF for feature collection.

Hooked function	Behavior
xfs_file_write_iter	Write
xfs_file_read_iter	Read
unlink, unlinkat	Delete
rename, renameat, renameat2	Rename
xfs_file_open	Open

as the prior work. Specifically, the total malicious-decision count is set to 5, the malicious-decision threshold is set to 0.8, the required number of logs is set to 150, and the event-aggregation time width is set to 370 ms.

Table 5 reports the detection performance of the constituent ML algorithm. The classifier was trained using the default settings of scikit-learn, and no additional hyperparameter tuning was performed. In the following experiments, the same set of detection models is used across all control methods so that observed differences can be attributed to delay-control mechanisms rather than detector variations.

The confusion matrix is shown in Table 6.

1) True Positive (TP): Number of malicious logs correctly classified as malicious.
2) True Negative (TN): Number of benign logs correctly classified as benign.
3) False Positive (FP): Number of benign logs incorrectly classified as malicious.
4) False Negative (FN): Number of malicious logs incorrectly classified as benign.

D. EXPERIMENT 1: SUSPENSION WINDOW DESIGN (FIXED-DURATION VERSUS BACKUP-COMPLETION WINDOW)

This experiment focuses on suspension-based delay control and evaluates how the choice of application window affects backup tracking. Suspension can directly suppress encryption progress. However, the duration for which the delay is maintained strongly influences both effectiveness and side effects, such as excessive blocking or premature release. Accordingly, this experiment compares a fixed-duration window and a backup-completion window to clarify which design yields more stable tracking improvements under ransomware behavioral diversity.

All experimental settings are kept identical across conditions, including the protected dataset, protected paths, open-trigger configuration, backup generation logic, and detector configuration. Only the application window of suspension-based control is varied as the independent variable. Upon an open event, the PID and PPID of the entity that issued the open are identified, and suspension control is applied to both. In the fixed-duration window, the PID/PPID is suspended immediately after open detection and resumed after a predetermined duration W_{pause}. In this study, W_{pause} is set to 3 s. In the backup-completion window, the PID/PPID is suspended immediately after open detection and resumed when the backup generation triggered by that open event completes. This design enables a direct comparison of the window-design trade-off: the fixed-duration window can be released too early (insufficient delay) or can retain unnecessary delay after backups complete, whereas the backup-completion window can secure the required delay adaptively but may increase the number of suspensions and the total suspended time.

E. EXPERIMENT 2: SCHEDULER LAYER (CFS VERSUS SCX)

This experiment isolates the effect of the scheduler layer on encryption throttling and tracking under CPU-share control. CPU-share control using cgroup v2 with cpu.weight reduces the CPU time allocated to the encryption process, thereby lowering encryption throughput. However, the realized allocation behavior can depend on the scheduler implementation. Therefore, this experiment fixes cpu.weight to 1 and fixes the application window to the backup-completion window, so that the scheduler layer (CFS vs SCX) becomes the primary independent variable.

In the CFS condition, upon open detection, the PID/PPID is moved to a dedicated cgroup, cpu.weight=1 is applied and maintained until backup completion, and then the process is moved back to the original cgroup. In the SCX condition, scx_flatcg is launched before the experiment to enable sched_ext, and the enabled state is kept fixed throughout the experiment. The same cgroup control as in the CFS condition (migration to a dedicated cgroup, cpu.weight=1, and release

TABLE 5. Algorithms used in machine learning and their detection performance.

Algorithm	Accuracy [%]	Precision [%]	Recall [%]	F1-score [%]	FPR	AUC
Random Forest (RF)	97.89	99.01	97.87	98.44	0.0206	0.9850

TABLE 6. Confusion matrix of the RF classifier.

		Actual Result	
		Positive	Negative
Prediction Result	Positive	1605	16
	Negative	35	760

upon backup completion) is applied after open detection. This design allows observed differences to be attributed primarily to the scheduler layer. As in the other experiments, the dataset, protected paths, open trigger, backup logic, and detector configuration are kept identical across conditions.

F. EXPERIMENT 3: CPU PARALLELISM THROTTLING (FIXED-DURATION VERSUS BACKUP-COMPLETION WINDOW)

This experiment evaluates CPU-parallelism throttling via cpuset and examines how the application window affects tracking. Modern ransomware can destroy a large number of files in a short time via highly parallel encryption. Restricting CPU parallelism can therefore serve as a direct lever to reduce the time scale of encryption progress. At the same time, the activation timing and duration of the restriction determine the trade-off between tracking improvement and performance impact. Accordingly, this experiment compares a fixed-duration window and a backup-completion window to clarify which design is more appropriate in practice.

In the fixed-duration window, the PID/PPID is restricted to a single CPU core immediately after open detection and reverted after a predetermined duration W_{cpu}. In the backup-completion window, the same one-core restriction is applied immediately after open detection and maintained until the corresponding backup generation completes, after which it is reverted. As in the other experiments, the dataset, protected paths, open trigger, backup logic, and detector configuration are kept identical across conditions. Only the application window of cpuset restriction is varied as the independent variable.

G. EXPERIMENT 4: BENIGN-WORKLOAD IMPACT

This experiment evaluates the impact of delay control on benign workloads and examines its implications for practical deployability. To this end, two legitimate encryption-related workloads were added in the same Linux experimental environment: file-level encryption using GPG and file copying to a LUKS-mounted encrypted volume. The latter serves as a representative benign file-copying workload. LUKS was selected because Linux kernel documentation describes it as the recommended interface for dm-crypt-based disk encryption, whereas GPG is widely used as a legitimate user-space tool for OpenPGP-based encryption and signing [40], [41].

During these runs, the detector and the associated control logic remained enabled under the same monitoring configuration as in the ransomware experiments. This design allows us to examine both whether benign workloads are falsely flagged and how much overhead the delay-control mechanisms introduce relative to the baseline condition without delay control. As in the other experiments, each condition was executed independently five times, and execution time was summarized using the mean and standard deviation.

H. METRICS

To separately evaluate backup feasibility and tracking of encrypted targets, this paper defines count-based quantities and ratio metrics based on unique base names. Auxiliary files corresponding to ransom notes (e.g., README-like files) are excluded from all counts.

Four count variables are defined as follows.

- N_E: the number of encrypted unique base names (encrypted file count).
- N_B: the number of unique base names for which backup creation succeeded (backup count).
- N_{EB}: the number of encrypted base names that have corresponding backups (encrypted-with-backup count).
- N_S: the number of unique base names that entered the backup processing pipeline (counted when targets are determined).

Here, N_S is counted when a backup target (targets) is determined by reconstructing the path from an open event and matching it against protect_list, and ROFBS begins processing that base name as a backup target.

Based on these counts, three ratio metrics are defined. They are reported as percentages; thus, a factor of 100 is applied when presenting results.

$$\mathrm{Cov}_B = \frac{N_B}{N_S} \tag{6}$$

Cov_B (Backup Coverage) represents the fraction of successful backups among attempts that entered processing.

$$\mathrm{EwB} = \frac{N_{EB}}{N_E} \tag{7}$$

EwB (Encrypted-with-Backup) is an intermediate metric representing the fraction of encrypted targets that are covered by corresponding backups. When $N_E = 0$, EwB is undefined due to division by zero and is treated as N/A.

$$\mathrm{Pre}_B = \frac{N_{EB}}{N_B} \tag{8}$$

Pre_B (Backup Precision) represents the fraction of generated backups that correspond to encrypted targets. When $N_B = 0$, Pre_B is undefined due to division by zero and is treated as N/A.

Taken together, these three metrics do not collapse backup effectiveness into a single value; rather, they separately capture feasibility, coverage, and precision. They should therefore be interpreted jointly with the absolute counts N_E, N_S, N_B, and N_{EB}. For example, a high Cov_B together with a low EwB indicates that backups are being generated but fail to keep pace with encrypted targets, whereas a low Pre_B indicates that many generated backups do not correspond to final encrypted outputs. Accordingly, the result tables are designed to be read using both the ratio metrics and the corresponding counts so that the dominant failure mode can be distinguished across control conditions.

VI. RESULTS

This section reports the results for ROFBS without delay control, which is treated as the baseline, and the three TSR experiments: Experiment VI-C (Suspension Window Design: fixed duration versus backup completion window), Experiment VI-D (Scheduler Layer: CFS versus SCX under cpu.weight=1), and Experiment VI-E (CPU Parallelism Throttling via cpuset: fixed duration versus backup completion window). It also reports the result of Experiment VI-F, which evaluates the impact on benign workloads using GPG and LUKS.

In all tables, N_E, N_S, N_B, and N_{EB} denote counts of unique base names, whereas Cov_B, EwB, and Pre_B are reported as percentages. When $N_E = 0$, EwB is reported as N/A because it is undefined. For all experimental conditions, five independent runs were conducted, and the values reported in the tables of this section are the means across runs. In addition, the standard deviations of the metrics are reported to show variability that cannot be captured by a single run. This makes the comparison across conditions more appropriate from the viewpoint of reproducibility. Although N_E, N_S, N_B, and N_{EB} are count based quantities defined on unique base names, they are reported here as averages across runs. Likewise, Cov_B, EwB, and Pre_B are reported together with their variability across independent runs.

A. PRELIMINARY SENSITIVITY ANALYSIS FOR PARAMETER SELECTION

To examine the sensitivity of the fixed parameter settings, a preliminary sensitivity analysis was conducted using Conti and Sodinokibi. For cpu.weight, three values were evaluated: 1, 10, and 50. For both W_{pause} and the suspend interval, three values were evaluated: 1 s, 3 s, and 10 s. When one parameter was varied, the remaining parameters were fixed at cpu.weight=1, $W_{\mathrm{pause}} = 3$ s, and suspend interval = 3 s.

The additional analysis showed that changing cpu.weight had only a limited effect. Within the present experimental range, CPU share control alone was not sufficient to reconstruct enough temporal slack. By contrast, changing the suspend interval had a larger effect, although the magnitude of the improvement depended on the ransomware family and was not monotonic.

TABLE 7. Sensitivity analysis (Counts): parameter settings for Conti and Sodinokibi.

	Conti				Sodinokibi			
Setting	N_E	N_S	N_B	N_{EB}	N_E	N_S	N_B	N_{EB}
cpu.weight=10	44	45	1	0	33	60	3	3
cpu.weight=50	44	46	2	1	34	62	3	3
$W_{\mathrm{pause}} = 1$ s	44	45	2	1	34	64	3	3
$W_{\mathrm{pause}} = 10$ s	44	45	1	0	33	60	3	3
Baseline CPU setting	45	46	1	0	36	63	4	4
Suspend interval = 1 s	38	39	5	4	34	59	4	4
Suspend interval = 10 s	9	10	6	5	30	49	5	5
Baseline suspend interval	9	9	6	5	42	55	12	12

TABLE 8. Sensitivity analysis (Metrics): parameter settings for Conti and Sodinokibi.

	Conti			Sodinokibi		
Setting	Cov_B[%]	EwB[%]	Pre_B[%]	Cov_B[%]	EwB[%]	Pre_B[%]
cpu.weight=10	2.22	0.00	0.00	5.00	9.09	100.00
cpu.weight=50	4.35	2.27	50.00	4.84	8.82	100.00
$W_{\mathrm{pause}} = 1$ s	4.44	2.27	50.00	4.69	8.82	100.00
$W_{\mathrm{pause}} = 10$ s	2.22	0.00	0.00	5.00	9.09	100.00
Baseline CPU setting	2.17	0.00	0.00	6.35	11.11	100.00
Suspend interval = 1 s	12.82	10.53	80.00	6.78	11.76	100.00
Suspend interval = 10 s	60.00	55.56	83.33	10.20	16.67	100.00
Baseline suspend interval	66.67	55.56	83.33	21.82	28.57	100.00

For Conti, a suspend interval of 3 s or longer substantially improved encrypted target coverage. For Sodinokibi, the highest EwB was obtained at 3 s, and no further improvement was obtained at 10 s. These results indicate that the effect of suspension is not determined solely by the duration itself, but also by when the first protected file open is observed, how strongly the suspension window overlaps with the initial burst of encryption, and how much progress the backup and detection processes can make during that interval.

Accordingly, the effectiveness of TSR should not be understood as a monotonic function of a single parameter. Rather, it should be understood as the result of the interaction between family specific encryption dynamics and system side dynamics. Based on this preliminary analysis, the main experiments were conducted with cpu.weight=1, $W_{\mathrm{pause}} = 3$ s, and suspend interval = 3 s.

B. BASELINE: ROFBS (NO DELAY CONTROL)

In the baseline condition, ROFBS performs backup creation upon open detection without applying any delay control to encryption progress (Table 3). This condition serves as the reference for assessing how well ROFBS can track encryption in the absence of TSR interventions. Table 9 summarizes the results.

For Conti, both Cov_B and EwB are approximately 90%, indicating that ROFBS can largely keep pace with encryption under this workload. In contrast, AvosLocker exhibits a large number of generated backups ($N_B = 42$) but only a small

TABLE 9. Baseline results (ROFBS without delay control, mean ± SD over five runs).

Family	N_E	N_S	N_B	N_{EB}	Cov_B [%]	EwB [%]	Pre_B [%]
Conti	20.4 ± 0.9	21.4 ± 0.9	18.6 ± 0.9	17.6 ± 0.9	87.00 ± 4.80	86.36 ± 5.02	94.61 ± 0.28
AvosLocker	43.4 ± 33.8	83.8 ± 39.2	42.4 ± 14.4	2.4 ± 1.9	57.91 ± 25.19	18.67 ± 24.87	6.57 ± 5.87
Sodinokibi	34.4 ± 6.2	60.4 ± 11.5	4.0 ± 1.0	4.0 ± 1.0	6.63 ± 1.14	11.60 ± 1.83	100.00 ± 0.00
HelloKitty	18.8 ± 4.3	60.0 ± 9.4	11.6 ± 0.9	0.0 ± 0.0	19.69 ± 3.18	0.00 ± 0.00	0.00 ± 0.00
IceFire	18.6 ± 10.6	27.2 ± 10.9	10.6 ± 1.7	2.6 ± 1.7	48.31 ± 29.22	14.41 ± 5.55	22.79 ± 13.87
Monti	22.0 ± 1.2	23.0 ± 1.2	17.4 ± 0.5	16.4 ± 0.5	75.89 ± 5.92	74.81 ± 6.13	94.25 ± 0.18
REvil	34.2 ± 1.9	52.0 ± 10.9	18.0 ± 9.7	0.2 ± 0.4	32.73 ± 11.34	0.56 ± 1.24	0.80 ± 1.79
LockBit	25.4 ± 3.0	25.4 ± 3.0	0.0 ± 0.0	0.0 ± 0.0	0.00 ± 0.00	0.00 ± 0.00	N/A

overlap with encrypted targets ($N_{EB} = 3$), resulting in low EwB $= 12.50\%$ and low $\mathrm{Pre}_B = 7.14\%$. This suggests that a substantial fraction of backup work is spent on files that do not end up corresponding to encrypted targets, which can amplify backlog growth under high-throughput behavior. For Sodinokibi, $N_B = 3$ and $\mathrm{Cov}_B = 5.17\%$ are both small, indicating that backup completion frequently fails to precede encryption/deletion. For HelloKitty, Cov_B remains non-negligible but $N_{EB} = 0$, yielding EwB $= 0\%$, which highlights a regime where backups are created but do not cover encrypted targets. IceFire shows a similarly low EwB, implying that ROFBS without delay control does not robustly maintain tracking under certain ransomware behaviors.

C. EXPERIMENT 1: SUSPENSION WINDOW DESIGN (FIXED-DURATION VERSUS BACKUP-COMPLETION WINDOW)

Experiment 1 evaluates how the application window design of suspension based control affects tracking. Tables 10 and 11 report the counts and the ratio metrics, respectively. Because suspension can affect both when encryption starts and how quickly the ransomware progresses, the absolute counts (N_E and N_S) may change across conditions. Therefore, the results are interpreted using both ratios and counts.

For Conti, the backup completion window yields better tracking than the fixed duration window. Specifically, the backup completion window achieves EwB $= 85.00\%$ with $N_{EB} = 17$ out of $N_E = 20$, whereas the fixed duration window yields EwB $= 55.56\%$ with $N_{EB} = 5$ out of $N_E = 9$. This indicates that maintaining suspension until backup completion can more consistently preserve encrypted target coverage for this family.

In contrast, Sodinokibi and IceFire show stronger results under the fixed duration window. For Sodinokibi, the fixed duration window yields $N_{EB} = 12$, whereas the backup completion window yields $N_{EB} = 3$, resulting in higher Cov_B and higher EwB under the fixed duration window. For IceFire, the fixed duration window yields EwB $= 50.00\%$ with $N_{EB} = 4$ out of $N_E = 8$, whereas the backup completion window yields EwB $= 21.43\%$ with $N_{EB} = 3$ out of $N_E = 14$. Monti shows the same tendency, with the fixed duration window achieving higher Cov_B and EwB than the backup completion window. These results suggest that, for certain ransomware behaviors, introducing a sufficiently strong delay early, even for a fixed duration, can be more effective than synchronizing the delay duration to backup completion. Although the absolute counts for AvosLocker vary across runs, the qualitative pattern is stable: overlap remains absent or negligible, and the observed artifacts remain more consistent with early suppression than with successful tracking.

AvosLocker exhibits $N_E = 0$ under the fixed duration window, which makes EwB undefined. Under this condition, backup files were still generated, but no encrypted outputs were observed. This pattern suggests that early intervention may have suppressed observable encryption progress before protected user data were encrypted. By contrast, under the backup completion window, AvosLocker shows large N_E but zero overlap, indicating that suspension tied only to backup completion is insufficient for maintaining encrypted target coverage against this family.

HelloKitty remains difficult under both windows, with $N_{EB} = 0$ throughout. Artifacts corresponding to README_TO_RESTORE and temporary files were observed, suggesting that the workflow progressed at least to an intermediate stage. However, a stable one to one correspondence between the objects captured as backup targets at open time and the final encrypted outputs could not be established. This pattern is more consistent with a target mapping mismatch than with simple backup lag.

REvil also remains difficult, with limited overlap under the fixed duration window and zero overlap under the backup completion window. LockBit shows $N_B = 0$ in both conditions, indicating that suspension alone did not restore backup feasibility for this sample.

D. EXPERIMENT 2: SCHEDULER LAYER (CFS VERSUS SCX)

Experiment 2 isolates the effect of the scheduler layer on CPU share based throttling by fixing cpu.weight=1 and applying control only within the backup completion window. Tables 12 and 13 report the results. Even under the same cpu.weight setting, substantial differences are observed between CFS and SCX in multiple samples.

The most prominent difference appears in Conti. Under CFS, backups almost never complete successfully, with $N_B = 1$ out of $N_S = 47$, yielding $\mathrm{Cov}_B = 2.13\%$ and EwB $= 0\%$.

TABLE 10. Experiment 1 (Counts): Suspend (fixed) vs Suspend (backup-completion window).

	fixed				backup-completion window			
Sample	N_E	N_S	N_B	N_{EB}	N_E	N_S	N_B	N_{EB}
Conti	9.4 ± 1.8	10.4 ± 1.8	6.8 ± 2.0	5.8 ± 2.0	20.0 ± 1.9	21.0 ± 1.9	18.2 ± 0.8	17.2 ± 0.8
AvosLocker	0.2 ± 0.4	18.4 ± 8.6	18.4 ± 8.6	0.2 ± 0.4	46.8 ± 24.7	96.6 ± 24.5	50.6 ± 0.9	0.8 ± 0.8
Sodinokibi	42.4 ± 9.3	55.6 ± 9.3	12.0 ± 7.3	12.0 ± 7.3	32.0 ± 8.3	56.0 ± 15.9	3.2 ± 1.1	3.2 ± 1.1
HelloKitty	4.8 ± 6.6	28.0 ± 10.1	11.2 ± 0.8	0.0 ± 0.0	23.4 ± 0.9	68.4 ± 2.2	10.8 ± 0.4	0.0 ± 0.0
IceFire	8.6 ± 2.6	15.2 ± 4.3	13.6 ± 3.4	7.2 ± 1.3	14.4 ± 8.3	21.8 ± 7.9	11.2 ± 2.5	3.8 ± 2.2
Monti	6.6 ± 1.5	7.6 ± 1.5	6.0 ± 0.7	5.0 ± 0.7	24.6 ± 3.8	25.6 ± 3.8	13.4 ± 4.9	12.4 ± 4.9
REvil	28.6 ± 1.8	48.8 ± 5.7	23.6 ± 5.9	3.4 ± 2.9	35.4 ± 3.2	51.4 ± 4.7	16.2 ± 4.1	0.2 ± 0.4
LockBit	22.0 ± 8.9	22.0 ± 8.9	0.0 ± 0.0	0.0 ± 0.0	27.2 ± 4.6	27.2 ± 4.6	0.0 ± 0.0	0.0 ± 0.0

TABLE 11. Experiment 1 (Metrics): Suspend (fixed) vs Suspend (backup-completion window).

	fixed			backup-completion window		
Sample	Cov_B [%]	EwB [%]	Pre_B [%]	Cov_B [%]	EwB [%]	Pre_B [%]
Conti	64.61 ± 9.99	60.65 ± 11.37	84.29 ± 4.29	87.29 ± 9.41	85.88 ± 10.17	94.50 ± 0.26
AvosLocker	100.00 ± 0.00	N/A	0.65 ± 1.44	55.62 ± 16.06	3.19 ± 4.09	1.56 ± 1.62
Sodinokibi	23.48 ± 18.13	26.64 ± 11.45	100.00 ± 0.00	6.05 ± 2.42	10.34 ± 3.56	100.00 ± 0.00
HelloKitty	43.19 ± 11.79	0.00 ± 0.00	0.00 ± 0.00	15.79 ± 0.55	0.00 ± 0.00	0.00 ± 0.00
IceFire	77.54 ± 24.26	70.47 ± 31.44	55.91 ± 25.00	58.19 ± 24.60	26.94 ± 5.74	31.97 ± 18.06
Monti	82.14 ± 20.82	79.84 ± 23.18	83.14 ± 2.04	54.62 ± 25.36	52.84 ± 26.23	91.24 ± 4.62
REvil	48.00 ± 8.29	12.03 ± 10.33	13.33 ± 9.65	31.30 ± 5.83	0.54 ± 1.21	0.91 ± 2.03
LockBit	0.00 ± 0.00	0.00 ± 0.00	N/A	0.00 ± 0.00	0.00 ± 0.00	N/A

Under SCX, backups succeed for essentially all observed targets, with $N_B = 19$ out of $N_S = 19$, and encrypted target coverage also becomes high, with $N_{EB} = 18$ out of $N_E = 18$, yielding $\mathrm{Cov}_B = 100.00\%$ and $\mathrm{EwB} = 100.00\%$. This indicates that the scheduler layer can be a decisive factor for TSR effectiveness when CPU share control is used.

Monti shows the same tendency. Under CFS, backups almost never complete successfully, with $\mathrm{Cov}_B = 2.17\%$ and $\mathrm{EwB} = 0\%$. Under SCX, both feasibility and encrypted target coverage become nearly perfect, yielding $\mathrm{Cov}_B = 100.00\%$ and $\mathrm{EwB} = 100.00\%$. Taken together, Conti and Monti suggest that scheduler selection can determine whether CPU share based TSR succeeds or collapses. For Conti and Monti, the contrast between CFS and SCX remains much larger than the observed run to run variation, which supports the robustness of the qualitative conclusion that scheduler selection can determine whether CPU share based TSR succeeds or collapses.

Sodinokibi also benefits from SCX, although the improvement is more moderate. Compared with CFS, SCX increases N_B from 5 to 15 and N_{EB} from 5 to 15, improving Cov_B from 7.81% to 35.71% and EwB from 13.51% to 40.54%. This suggests that, when encryption throughput is strongly shaped by CPU contention, the effective mapping from cgroup weight to CPU allocation can translate into improved tracking.

In contrast, AvosLocker remains difficult to protect under CPU share control in both scheduler settings. Even under SCX, overlap is extremely small, with $N_{EB} = 1$ out of $N_E = 75$, yielding $\mathrm{EwB} = 1.33\%$ and $\mathrm{Pre}_B = 1.96\%$. HelloKitty also remains at $N_{EB} = 0$ under both schedulers. REvil shows only limited improvement under SCX, with EwB increasing from 0% to 4.44%. LockBit remains unrecoverable in both settings, with $N_B = 0$ throughout.

IceFire exhibits a different pattern. Under SCX, Cov_B increases to 85.71%, but EwB remains 0% because no overlap with encrypted targets is recovered. This indicates that improved backup feasibility does not necessarily imply improved encrypted target coverage.

Overall, these results show that identical cgroup weight settings can lead to very different outcomes depending on the scheduler layer. At the same time, the effect is not uniform across samples, indicating that scheduler based throttling is effective only when the dominant bottleneck is aligned with the selected control lever.

E. EXPERIMENT 3: CPU PARALLELISM THROTTLING (FIXED-DURATION VERSUS BACKUP-COMPLETION WINDOW)

Experiment 3 evaluates CPU parallelism throttling via cpuset and compares the fixed duration window with the backup completion window. Tables 14 and 15 report the results. Overall, cpuset based throttling shows mixed effectiveness across samples, and the choice of application window does not yield uniform improvements.

For Conti, both Cov_B and EwB remain low under both window designs. The fixed duration window yields slightly

TABLE 12. Experiment 2 (Counts): Scheduling (CFS + cgroup) vs Scheduling (SCX + scx_flatcg + cgroup).

	CFS + cgroup				SCX + scx_flatcg + cgroup			
Sample	N_E	N_S	N_B	N_{EB}	N_E	N_S	N_B	N_{EB}
Conti	46.4 ± 2.1	47.4 ± 2.1	1.4 ± 0.5	0.4 ± 0.5	18.0 ± 0.0	19.0 ± 0.0	19.0 ± 0.0	18.0 ± 0.0
AvosLocker	68.6 ± 3.6	115.6 ± 4.0	47.0 ± 4.2	0.0 ± 0.0	75.4 ± 32.9	125.4 ± 32.9	51.2 ± 0.8	1.2 ± 0.8
Sodinokibi	37.0 ± 0.0	64.0 ± 0.0	5.0 ± 0.0	5.0 ± 0.0	37.2 ± 5.7	42.0 ± 6.3	15.8 ± 3.1	15.8 ± 3.1
HelloKitty	21.2 ± 1.3	65.0 ± 2.8	11.2 ± 0.8	0.0 ± 0.0	12.0 ± 0.0	48.0 ± 0.0	12.0 ± 0.0	0.0 ± 0.0
IceFire	18.2 ± 9.0	26.4 ± 8.7	11.2 ± 2.3	2.8 ± 2.5	1.8 ± 4.0	7.6 ± 4.9	6.2 ± 2.5	0.6 ± 1.3
Monti	45.2 ± 1.3	46.2 ± 1.3	1.4 ± 0.9	0.4 ± 0.9	18.0 ± 0.0	19.0 ± 0.0	19.0 ± 0.0	18.0 ± 0.0
REvil	38.4 ± 4.5	59.0 ± 11.6	20.6 ± 7.7	0.0 ± 0.0	45.6 ± 14.1	71.8 ± 13.5	28.6 ± 6.8	2.4 ± 1.5
LockBit	26.2 ± 2.8	26.2 ± 2.8	0.0 ± 0.0	0.0 ± 0.0	6.0 ± 0.0	6.2 ± 0.4	0.0 ± 0.0	0.0 ± 0.0

TABLE 13. Experiment 2 (Metrics): Scheduling (CFS + cgroup) vs Scheduling (SCX + scx_flatcg + cgroup).

	CFS + cgroup			SCX + scx_flatcg + cgroup		
Sample	Cov_B [%]	EwB [%]	Pre_B [%]	Cov_B [%]	EwB [%]	Pre_B [%]
Conti	2.94 ± 1.10	0.84 ± 1.16	20.00 ± 27.39	100.00 ± 0.00	100.00 ± 0.00	94.74 ± 0.00
AvosLocker	40.63 ± 3.06	0.00 ± 0.00	0.00 ± 0.00	44.37 ± 16.93	1.33 ± 0.91	2.32 ± 1.60
Sodinokibi	7.81 ± 0.00	13.51 ± 0.00	100.00 ± 0.00	37.61 ± 5.23	42.25 ± 3.29	100.00 ± 0.00
HelloKitty	17.23 ± 1.10	0.00 ± 0.00	0.00 ± 0.00	25.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
IceFire	48.41 ± 23.99	20.81 ± 24.39	22.83 ± 15.18	89.64 ± 16.39	N/A	6.00 ± 13.42
Monti	3.08 ± 2.09	0.93 ± 2.08	13.33 ± 29.81	100.00 ± 0.00	100.00 ± 0.00	94.74 ± 0.00
REvil	34.03 ± 6.26	0.00 ± 0.00	0.00 ± 0.00	40.43 ± 8.74	5.03 ± 2.42	8.65 ± 6.08
LockBit	0.00 ± 0.00	0.00 ± 0.00	N/A	0.00 ± 0.00	0.00 ± 0.00	N/A

better overlap than the backup completion window, with EwB $= 2.33\%$ versus 0.00%, but the overall effect remains limited. For Sodinokibi, both designs remain low in Cov_B, and the backup completion window yields slightly higher encrypted target coverage than the fixed duration window, with EwB $= 11.11\%$ versus 8.82%.

AvosLocker remains difficult in both settings. Although backups are still generated in large numbers, no overlap with encrypted targets is recovered, and both EwB and Pre_B remain at or near zero. HelloKitty also remains at $N_{EB} = 0$ under both windows, indicating that cpuset throttling alone does not recover encrypted target coverage for this sample. REvil shows the same pattern, with no overlap recovered under either window. LockBit remains unrecoverable in both conditions, with $N_B = 0$ throughout.

IceFire and Monti show different tendencies. For IceFire, the fixed duration window yields a small amount of overlap, with EwB $= 10.00\%$, whereas the backup completion window yields EwB $= 0.00\%$. For Monti, the backup completion window clearly outperforms the fixed duration window, improving Cov_B from 4.44% to 12.50% and EwB from 2.27% to 10.26%. These results indicate that CPU parallelism throttling can improve coverage for specific samples, but it is not a uniformly effective TSR lever across families.

F. EXPERIMENT 4: IMPACT ON BENIGN WORKLOADS

Experiment 4 evaluates the impact of delay control on benign workloads. The benign workloads considered are file copying to a LUKS mounted encrypted volume and file level encryption using GPG. Tables 16 and 17 report the execution time, the relative overhead with respect to the baseline, and the completion status for the two workloads. Both workloads completed under all conditions.

Suspend (fixed) imposes by far the largest overhead on both workloads. For the LUKS workload, the execution time reaches 1980.184 ± 361.222 s, and for the GPG workload it reaches 6076.710 ± 0.333 s, corresponding to slowdowns of 3210.24% and 5630.04%, respectively, relative to the baseline. By contrast, Suspend (backup completion window) remains close to the baseline on the LUKS workload and introduces only a relatively small overhead on the GPG workload.

Scheduling (CFS + cgroup) remains close to the baseline on the LUKS workload but introduces a slowdown of 57.27% on the GPG workload. By contrast, Scheduling (SCX + scx_flatcg + cgroup) yields a slowdown of 35.56% on the LUKS workload and 14.92% on the GPG workload, indicating a moderate overhead on both workloads. These results show that the effect of scheduler based control depends strongly on both the specific method and the workload.

CPU parallelism control also shows different behavior depending on the configuration. CPU parallelism (cpuset, backup completion window) yields a slowdown of 50.90% on the LUKS workload and 7.33% on the GPG workload. By contrast, CPU parallelism (cpuset, fixed) remains close to the baseline on the LUKS workload and introduces only a small slowdown of 6.53% on the GPG workload.

TABLE 14. Experiment 3 (Counts): CPU parallelism (cpuset, fixed) vs CPU parallelism (cpuset, backup-completion window).

	cpuset, fixed				cpuset, backup-completion window			
Sample	N_E	N_S	N_B	N_{EB}	N_E	N_S	N_B	N_{EB}
Conti	43.4 ± 4.2	44.4 ± 4.2	2.6 ± 2.5	1.6 ± 2.5	45.4 ± 1.3	46.4 ± 1.3	1.4 ± 0.9	0.4 ± 0.9
AvosLocker	66.2 ± 6.1	116.0 ± 6.0	50.4 ± 1.5	0.6 ± 1.3	67.2 ± 5.6	115.8 ± 6.2	48.8 ± 3.3	0.2 ± 0.4
Sodinokibi	34.8 ± 2.8	61.6 ± 3.3	3.8 ± 1.6	2.8 ± 2.2	36.6 ± 0.5	63.6 ± 0.9	4.8 ± 0.4	4.8 ± 0.4
HelloKitty	21.4 ± 3.6	63.6 ± 7.4	10.4 ± 0.9	0.0 ± 0.0	23.6 ± 2.5	69.4 ± 5.2	11.2 ± 0.4	0.0 ± 0.0
IceFire	10.8 ± 13.1	18.6 ± 13.3	9.2 ± 1.8	1.4 ± 1.5	4.6 ± 6.3	12.0 ± 6.9	7.4 ± 1.7	0.8 ± 1.1
Monti	44.0 ± 1.6	45.0 ± 1.6	2.4 ± 1.1	1.4 ± 1.1	39.4 ± 10.3	40.4 ± 10.3	5.4 ± 7.2	4.4 ± 7.2
REvil	37.8 ± 6.1	60.2 ± 7.1	22.4 ± 7.7	0.0 ± 0.0	33.8 ± 2.7	58.0 ± 8.8	24.2 ± 6.8	0.0 ± 0.0
LockBit	25.0 ± 1.2	25.0 ± 1.2	0.0 ± 0.0	0.0 ± 0.0	22.6 ± 1.7	22.4 ± 1.5	0.0 ± 0.0	0.0 ± 0.0

TABLE 15. Experiment 3 (Metrics): CPU parallelism throttling using cpuset (fixed vs backup-completion window).

	cpuset, fixed			cpuset, backup-completion window		
Sample	Cov_B[%]	EwB[%]	Pre_B[%]	Cov_B[%]	EwB[%]	Pre_B[%]
Conti	6.39 ± 7.09	4.23 ± 7.04	37.14 ± 36.91	3.07 ± 2.10	0.93 ± 2.08	13.33 ± 29.81
AvosLocker	43.54 ± 2.45	0.88 ± 1.97	1.13 ± 2.53	42.18 ± 2.64	0.27 ± 0.60	0.39 ± 0.88
Sodinokibi	6.07 ± 2.53	7.88 ± 5.86	80.00 ± 44.72	7.55 ± 0.74	13.11 ± 1.13	100.00 ± 0.00
HelloKitty	16.51 ± 2.17	0.00 ± 0.00	0.00 ± 0.00	16.20 ± 1.23	0.00 ± 0.00	0.00 ± 0.00
IceFire	68.35 ± 33.53	16.35 ± 7.51	13.13 ± 13.68	72.71 ± 25.18	17.42 ± 1.07	9.00 ± 12.45
Monti	5.41 ± 2.74	3.26 ± 2.72	48.33 ± 29.11	19.94 ± 32.90	18.00 ± 33.41	44.89 ± 44.00
REvil	36.85 ± 10.01	0.00 ± 0.00	0.00 ± 0.00	41.05 ± 6.43	0.00 ± 0.00	0.00 ± 0.00
LockBit	0.00 ± 0.00	0.00 ± 0.00	N/A	0.00 ± 0.00	0.00 ± 0.00	N/A

TABLE 16. Execution time overhead on the benign LUKS workload.

Method	Execution Time (s)	Slowdown vs. Baseline (%)	Completed
Baseline	59.820 ± 6.581	0.00	Yes
Suspend (fixed)	1980.184 ± 361.222	3210.24	Yes
Suspend (backup completion window)	56.868 ± 5.578	-4.93	Yes
Scheduling (CFS + cgroup)	57.918 ± 8.210	-3.18	Yes
Scheduling (SCX + scx_flatcg + cgroup)	81.090 ± 12.887	35.56	Yes
CPU parallelism (cpuset, backup completion window)	90.268 ± 6.291	50.90	Yes
CPU parallelism (cpuset, fixed)	59.282 ± 7.031	-0.90	Yes

TABLE 17. Execution time overhead on the benign GPG workload.

Method	Execution Time (s)	Slowdown vs. Baseline (%)	Completed
Baseline	106.050 ± 8.798	0.00	Yes
Suspend (fixed)	6076.710 ± 0.333	5630.04	Yes
Suspend (backup completion window)	116.958 ± 8.107	10.29	Yes
Scheduling (CFS + cgroup)	166.784 ± 14.301	57.27	Yes
Scheduling (SCX + scx_flatcg + cgroup)	121.870 ± 9.971	14.92	Yes
CPU parallelism (cpuset, backup completion window)	113.828 ± 3.784	7.33	Yes
CPU parallelism (cpuset, fixed)	112.980 ± 3.800	6.53	Yes

VII. DISCUSSION

This section discusses when and why time scale reconfiguration is effective, using ROFBS without delay control as the reference (Table 9). The discussion is organized along four experimental axes: suspension based timing intervention (Experiment 1), scheduler layer effects under CPU share control (Experiment 2), CPU parallelism throttling via cpuset (Experiment 3), and impact on benign workloads (Experiment 4). In addition to the ratio metrics Cov_B, EwB, and Pre_B, the absolute counts N_E, N_S, N_B, and N_{EB} are emphasized. This is important because the same ratio can arise from qualitatively different regimes, such as cases where encryption barely progresses (N_E is small) and cases where encryption progresses at scale (N_E is large).

Here, Cov_B captures backup completion feasibility relative to observed backup targets, EwB captures encrypted-target

coverage, and Pre_B captures backup precision. By jointly interpreting ratios and counts, it becomes possible to distinguish (i) situations where ROFBS-TSR is effective because encryption is slowed or suppressed, from (ii) situations where backups are produced but do not correspond to encrypted targets, and from (iii) situations where the defense cannot keep up at all under Fundamental Time-Scale Mismatch. Because the reported values are averages over five independent runs, the following discussion emphasizes qualitative tendencies that remain stable across runs. Where the standard deviations are relatively large, the interpretation should be understood as indicating a tendency rather than a strict ranking among methods.

A. EXPERIMENT 1: SUSPENSION WINDOW DESIGN (FIXED DURATION VERSUS BACKUP COMPLETION WINDOW)

Experiment 1 compares two window designs for suspension based delay control (Counts: Table 10; Metrics: Table 11). The results indicate that the effectiveness of suspension depends not only on whether suspension is applied, but also on how much delay is guaranteed in the early phase, what the first observed open events correspond to, and how continuously and aggressively encryption proceeds afterward. In other words, the window design determines whether temporal slack is created at the critical moment where Fundamental Time Scale Mismatch would otherwise dominate.

Conti illustrates the advantage of maintaining suspension until backup completion when encryption progresses continuously. Under the fixed duration window, $N_E = 9.4 \pm 1.8$, $N_B = 6.8 \pm 2.0$, and $N_{EB} = 5.8 \pm 2.0$, yielding $\mathrm{Cov}_B = 64.61 \pm 9.99\%$ and $\mathrm{EwB} = 60.65 \pm 11.37\%$. Under the backup completion window, $N_E = 20.0 \pm 1.9$, $N_B = 18.2 \pm 0.8$, and $N_{EB} = 17.2 \pm 0.8$, yielding $\mathrm{Cov}_B = 87.29 \pm 9.41\%$ and $\mathrm{EwB} = 85.88 \pm 10.17\%$. This indicates that, for a family such as Conti, keeping suspension active until backup completion can more reliably preserve temporal slack and maintain encrypted target coverage.

In contrast, Sodinokibi, IceFire, and Monti show stronger results under the fixed duration window. For Sodinokibi, the fixed duration window yields $\mathrm{Cov}_B = 23.48 \pm 18.13\%$ and $\mathrm{EwB} = 26.64 \pm 11.45\%$, whereas the backup completion window yields $\mathrm{Cov}_B = 6.05 \pm 2.42\%$ and $\mathrm{EwB} = 10.34 \pm 3.56\%$. For IceFire, the fixed duration window yields $\mathrm{EwB} = 70.47 \pm 31.44\%$, whereas the backup completion window yields $\mathrm{EwB} = 26.94 \pm 5.74\%$. Monti shows the same tendency, with the fixed duration window achieving $\mathrm{Cov}_B = 82.14 \pm 20.82\%$ and $\mathrm{EwB} = 79.84 \pm 23.18\%$, compared with $\mathrm{Cov}_B = 54.62 \pm 25.36\%$ and $\mathrm{EwB} = 52.84 \pm 26.23\%$ under the backup completion window. These results suggest that, for certain ransomware behaviors, introducing a sufficiently strong delay early, even for a fixed duration, can be more effective than synchronizing the delay duration to backup completion.

Although the absolute counts for AvosLocker vary across runs, the qualitative pattern is stable: overlap remains absent or negligible, and the observed artifacts remain more consistent with early suppression than with successful tracking. AvosLocker exhibits $N_E = 0$ under the fixed duration window, which makes EwB undefined. At the same time, backups are still created ($N_B = 18$), but none correspond to encrypted targets ($N_{EB} = 0$, and thus $\mathrm{Pre}_B = 0\%$). Prior analysis reports that AvosLocker places README_FOR_RESTORE, then launches encryption threads, and finally appends the.avoslinux extension [42]. In our inspection under the fixed duration window, README_FOR_RESTORE was present and both original files and backup files remained, whereas no files with the.avoslinux extension were observed. This pattern is more consistent with delay control taking effect during the early stage between ransom note placement and thread based encryption than with simple backup lag. By contrast, under the backup completion window, AvosLocker shows large N_E but zero overlap, indicating that suspension tied only to backup completion is insufficient for maintaining encrypted target coverage against this family.

HelloKitty remains difficult under both windows, with $N_{EB} = 0$ throughout. Artifacts corresponding to README_TO_RESTORE and temporary files were observed, suggesting that the workflow progressed at least to an intermediate stage. Unit 42 and VMware analyses also suggest that HelloKitty introduces sample specific filename transitions involving temporary files and ransom note related outputs before the final encrypted files are established [43], [44]. However, a stable one to one correspondence between the objects captured as backup targets at open time and the final encrypted outputs could not be established in our observation. Taken together, this pattern is more consistent with a family specific target mapping mismatch than with simple backup lag.

REvil also remains difficult, with limited overlap under the fixed duration window and zero overlap under the backup completion window. LockBit shows $N_B = 0$ in both conditions, indicating that suspension alone did not restore backup feasibility for this sample.

Overall, Experiment 1 shows that suspension is a powerful TSR lever, but the optimal window design is not universal. Whether the fixed duration window or the backup completion window is preferable depends on ransomware behavior, the nature of the first observed open events, and the distribution of backup completion times. This indicates that TSR should be treated as a control design problem rather than as a one size fits all mechanism.

B. EXPERIMENT 2: SCHEDULER LAYER (CFS VERSUS SCX)

Experiment 2 compares CPU share based throttling under two scheduler layers, CFS and SCX, while fixing cpu.weight=1 and using the backup completion window (Counts: Table 12; Metrics: Table 13). The results demonstrate that identical

cgroup weight configurations can lead to substantially different effective throttling, implying that the scheduler layer can be a decisive factor in whether temporal slack is actually reconstructed in practice.

The clearest evidence appears in Conti and Monti. For both samples, CFS leads to near total failure of backup completion, whereas SCX produces almost complete recovery of both feasibility and encrypted target coverage. In Conti, under CFS, $N_E = 46.4 \pm 2.1$, $N_S = 47.4 \pm 2.1$, $N_B = 1.4 \pm 0.5$, and $N_{EB} = 0.4 \pm 0.5$, yielding $\mathrm{Cov}_B = 2.94 \pm 1.10\%$ and $\mathrm{EwB} = 0.84 \pm 1.16\%$. Under SCX, $N_E = 18.0 \pm 0.0$, $N_S = 19.0 \pm 0.0$, $N_B = 19.0 \pm 0.0$, and $N_{EB} = 18.0 \pm 0.0$, yielding $\mathrm{Cov}_B = 100.00 \pm 0.00\%$ and $\mathrm{EwB} = 100.00 \pm 0.00\%$. Monti shows the same transition, from near total failure under CFS to near complete coverage under SCX. For Conti and Monti, the contrast between CFS and SCX remains much larger than the observed run to run variation, which supports the robustness of the qualitative conclusion that scheduler selection can determine whether CPU share based TSR succeeds or collapses.

Sodinokibi also benefits from SCX, although less dramatically. Compared with CFS, SCX improves both Cov_B and EwB while maintaining $\mathrm{Pre}_B = 100.00 \pm 0.00\%$. This suggests that, when encryption throughput is sufficiently affected by CPU contention, the scheduler layer can change how cgroup weight is translated into practical throttling and tracking performance.

In contrast, AvosLocker, HelloKitty, REvil, and LockBit remain difficult under CPU share control even when SCX is used. AvosLocker still shows almost no overlap, HelloKitty remains at $N_{EB} = 0$, REvil improves only slightly, and LockBit shows no successful backups at all. These cases indicate that resource allocation alone is insufficient when encryption progress is not primarily constrained by CPU share, or when the dominant mismatch arises from target mapping rather than timing.

IceFire provides an instructive example of why the ratio metrics must be interpreted jointly. Under SCX, Cov_B rises to $89.64 \pm 16.39\%$, indicating that backup completion becomes feasible for most observed targets. However, EwB remains undefined or near zero across runs, showing that this increase in feasibility does not translate into stable coverage of encrypted targets. This is consistent with a regime in which backup generation improves but overlap with the final encrypted outputs is not recovered.

It is also important to note that SCX is enabled globally in this experiment. Therefore, the observed difference between CFS and SCX should not be interpreted solely as a ransomware side throttling effect. The backup path and detector related processing may also receive different scheduling service under SCX than under CFS. Accordingly, the observed gap should be interpreted as an end to end scheduler layer effect under the present setup rather than as a purely ransomware side effect.

Overall, Experiment 2 shows that scheduler based throttling can be highly effective, but only for a subset of samples. Its success depends on whether CPU allocation is the dominant bottleneck and on how the scheduler layer affects the end to end balance between encryption progress, backup completion, and detector progress.

C. EXPERIMENT 3: CPU PARALLELISM THROTTLING (FIXED-DURATION VERSUS BACKUP-COMPLETION WINDOW)

Experiment 3 evaluates CPU parallelism throttling using cpuset and compares the fixed duration window with the backup completion window (Counts: Table 14; Metrics: Table 15). The results show that limiting CPU parallelism can improve coverage in specific cases, but it is not a universally reliable TSR lever across samples.

For Conti, cpuset throttling remains largely ineffective. Both window designs yield very low feasibility and coverage, and the backup completion window reduces overlap to zero. This suggests that restricting CPU concurrency does not reconstruct enough temporal slack when the dominant constraints are not governed primarily by CPU core parallelism.

Sodinokibi shows only limited recovery under cpuset control. The backup completion window performs slightly better than the fixed duration window, but both remain far below the level required for robust tracking. This indicates that restricting parallelism alone is insufficient to overcome the dominant bottleneck in this sample.

AvosLocker, HelloKitty, REvil, and LockBit remain difficult under both cpuset settings. AvosLocker still produces many backups but no overlap, HelloKitty remains at $N_{EB} = 0$, REvil also remains at zero overlap, and LockBit shows no successful backups at all. Taken together, these cases indicate that CPU parallelism control alone does not recover tracking when the dominant bottleneck is related to target mapping, file replacement behavior, or backup side feasibility rather than CPU concurrency.

IceFire and Monti provide useful contrasts. For IceFire, the fixed duration window yields a small amount of overlap, whereas the backup completion window yields none. For Monti, by contrast, the backup completion window substantially improves both feasibility and encrypted target coverage. These differences suggest that the usefulness of cpuset depends strongly on whether concurrency is actually a dominant driver of damage expansion in the evaluated sample. At the same time, the effect of cpuset is less stable than the scheduler layer effect observed in Experiment 2. In particular, the apparent improvement for some samples should be interpreted as conditional rather than uniformly reproducible, because the run to run variation is larger under cpuset based control.

Overall, Experiment 3 shows that cpuset provides a direct handle on concurrency, but its effectiveness depends on the ransomware family and on whether CPU parallelism is the dominant factor shaping encryption progress. Accordingly, cpuset should be treated as a conditional control option rather than as a universally effective TSR mechanism.

D. EXPERIMENT 4: IMPACT ON BENIGN WORKLOADS

The results of Experiment 4 show that the operational side effects on benign workloads depend strongly on both the delay control method and the workload type. As shown in Tables 16 and 17, both the LUKS workload and the GPG workload completed under all conditions, and no case was observed in which the benign task became unable to finish within the scope of this evaluation. At the same time, the execution time varied substantially across conditions, and Suspend (fixed) imposed by far the largest overhead on both workloads. This indicates that a configuration that keeps strong suspension based control continuously enabled is difficult to justify in practice from the standpoint of its impact on benign workloads.

By contrast, Suspend (backup completion window) remained close to the baseline on the LUKS workload and introduced only a relatively small overhead on the GPG workload. This suggests that, even within suspension based control, limiting the control duration according to actual backup progress can substantially reduce the operational side effects.

The scheduler based methods showed different tendencies. Scheduling (CFS + cgroup) remained close to the baseline on the LUKS workload, but introduced a comparatively large overhead on the GPG workload. By contrast, Scheduling (SCX + scx_flatcg + cgroup) produced a moderate overhead on both workloads, indicating that the effect of the scheduler layer on benign workloads is also dependent on the workload. This result suggests that the practical usefulness of scheduler based control cannot be judged only from its ransomware side mitigation effect and should also be evaluated in terms of its side effects on benign workloads.

CPU parallelism control also showed different behavior depending on the application window. CPU parallelism (cpuset, fixed) remained relatively light for both workloads, whereas CPU parallelism (cpuset, backup completion window) imposed a larger overhead on the LUKS workload. This suggests that the cost of restricting parallelism is not uniform, but varies with both the duration of the restriction and the nature of the workload.

Overall, from the standpoint of benign workloads, the practical deployability of TSR cannot be determined by any single control method in a uniform way. For normal operation, methods such as Suspend (backup completion window) and relatively light CPU parallelism control remain plausible candidates, whereas stronger control such as Suspend (fixed) is more realistically treated as an escalated response during high risk periods or after a malicious decision is reached. However, a quantitative evaluation of such staged operational policies is beyond the scope of this paper and remains future work.

E. LIMITATIONS

The proposed mechanism itself is not tied to a virtual machine based deployment and relies only on standard Linux kernel mechanisms. However, the present evaluation was conducted on an AlmaLinux guest running on VirtualBox over a Windows 11 host. Because the measurements were obtained in a hosted virtual machine environment, the influence of the host side virtualization stack on sched_ext behavior, eBPF hook latency, and the observed difference between CFS and SCX cannot be completely excluded. Accordingly, the scheduler layer comparison reported in this paper should be interpreted as an observation obtained under the present virtualized setup, and replication on bare metal hardware remains future work.

In Experiment 2, scx_flatcg remained enabled throughout the experiment. Therefore, the observed difference between CFS and SCX should not be interpreted solely as a throttling effect on the ransomware process. The backup process and detector related processing may also receive different scheduling service under SCX than under CFS. Accordingly, the observed gap should be interpreted as an end to end scheduler layer effect under the current setup. The present experiment does not fully separate the extent to which the gap is caused by stronger suppression of encryption from the extent to which it is caused by different scheduler service for backup and detection. A more isolated evaluation of the backup path remains future work.

F. OVERALL IMPLICATIONS

Across all experiments, the key to improving tracking is early and sufficiently strong intervention that prevents Fundamental Time-Scale Mismatch from becoming self-amplifying. Suspension-based timing intervention can be highly effective, but its window design determines whether temporal slack is reliably reconstructed or prematurely released. Resource-based control can also be effective, but the scheduler layer can critically affect how control parameters translate into actual throttling. CPU-parallelism throttling can help in concurrency-dominated cases, yet it is not universally effective.

Finally, the persistent cases with low or zero overlap ($N_{EB} = 0$) despite nonzero backups highlight that, beyond slowing encryption, it is equally important that the trigger-to-target mapping aligns backed-up files with encrypted outcomes. This observation supports the use of the three metrics together with the absolute counts: they make it possible to separate feasibility failures, timing failures, and relevance failures, and thus to locate the dominant bottleneck under each ransomware behavior.

The results also indicate that, although the effectiveness of TSR is not uniform across ransomware samples, the design principle of reconstructing the time scale through protected file open observation and delay control retains a certain degree of generalizability to unseen or newly introduced ransomware behaviors. However, its effectiveness depends on whether early intervention is actually achieved and on whether the dominant bottleneck of each sample is aligned with the selected control lever. Moreover, the evidence for unseen samples in this paper is conditioned on the fact that

the detector successfully reached a stop decision for samples such as Monti, REvil, and LockBit, which were not included in detector training. Accordingly, the present study does not evaluate the case in which the detector fails to identify such samples. For this reason, the results should not be interpreted as evidence of universal zero day protection, but rather as evidence that clarifies both the effectiveness and the limits of TSR under a broader set of conditions that includes unknown samples.

VIII. CONCLUSION

This paper presented Time Scale Reconfiguration, or TSR, as a framework for reconstructing conditions under which real time backup can keep pace with ransomware encryption by intervening in the time scale of encryption progress. TSR is built on the Real time Open File Backup System, or ROFBS, which creates backups upon file open events. By formulating the temporal conflict between encryption progress and backup completion as Fundamental Time Scale Mismatch, the framework aims not to rely only on faster backup, but to reconstruct temporal slack by delaying the attack side.

To realize this framework, the paper implemented ROFBS TSR by starting from file open observation via eBPF and combining process suspension, CPU share control via cgroup v2, scheduler layer control via sched_ext, and CPU parallelism control via cpuset. For evaluation, the paper introduced Cov_B for backup completion feasibility, EwB for encrypted target coverage, and Pre_B for backup precision, and interpreted these ratios together with the corresponding absolute counts so that different failure modes could be distinguished.

The evaluation covered multiple Linux ransomware samples, including Sodinokibi, IceFire, HelloKitty, AvosLocker, Conti, Monti, REvil, and LockBit, and all conditions were compared using the mean and standard deviation over five independent runs. The results show, first, that suspension based control is a strong TSR lever, but the relative advantage of the fixed duration window and the backup completion window depends on the sample, indicating that both strong early delay and sustained protection until backup completion can matter. Second, even under the same cpu.weight, the scheduler layer can substantially change tracking performance, with SCX producing decisive improvements for samples such as Conti and Monti, while remaining much less effective for others. Third, CPU parallelism control via cpuset is useful only for a subset of samples and should not be treated as a uniformly effective mechanism; its benefit appears only when CPU concurrency is a dominant bottleneck. Fourth, the benign workload evaluation shows that continuously enabled strong suspension imposes extremely high overhead, whereas lighter controls can remain near the baseline for some workloads. In addition, the sensitivity analysis indicates that changing cpu.weight has only limited effect within the present range, whereas the suspend interval has a larger impact, although its usefulness remains dependent on the sample.

Taken together, these findings indicate that TSR should not be understood as a single control mechanism that works uniformly across ransomware samples. Rather, its effectiveness depends on the interaction among ransomware behavior, trigger timing, backup side progress, and the timing at which detection becomes effective. Moreover, although the study shows a certain degree of generalizability under conditions that include unseen samples, that conclusion is conditioned on the detector successfully reaching a stop decision for those samples. Accordingly, the present work does not claim universal zero day protection, but instead clarifies both the effectiveness and the limits of TSR under a broader set of conditions that includes unknown samples.

Several directions remain for future work. These include adaptive TSR that adjusts the window design and delay magnitude according to observed signals, re evaluation on bare metal hardware, and a more isolated analysis of scheduler layer effects on the backup path. It is also necessary to extend the quantitative evaluation of side effects on benign workloads and overall system throughput in order to derive practical operational guidelines that balance security and availability.

REFERENCES

[1] M. Anghel and A. Racautanu, "A note on different types of ransomware attacks," *Cryptology ePrint Archive*, Paper 2019/605, Jun. 2019. [Online]. Available: https://eprint.iacr.org/2019/605

[2] CyberEdge. (2023). *Cyberedge Group: Report Defense Cyberthreat 2023*. [Online]. Available: https://cyberedgegroup.com/cdr/

[3] Allianz. (2023). *Allianz Commercial Cyber Security Trends 2023*. [Online]. Available: https://commercial.allianz.com/news-and-insights/reports/cyber-security-trends-2023.html

[4] A. H. Celdrán, P. M. S. Sánchez, M. A. Castillo, G. Bovet, G. M. Pérez, and B. Stiller, "Intelligent and behavioral-based detection of malware in IoT spectrum sensors," *Int. J. Inf. Secur.*, vol. 22, no. 3, pp. 541–561, Jun. 2023.

[5] A. Alraizza and A. Algarni, "Ransomware detection using machine learning: A survey," *Big Data Cognit. Comput.*, vol. 7, no. 3, p. 143, 2023.

[6] C. Beaman, A. Barkworth, T. D. Akande, S. Hakak, and M. K. Khan, "Ransomware: Recent advances, analysis, challenges and future research directions," *Comput. Secur.*, vol. 111, Dec. 2021, Art. no. 102490.

[7] S. H. Kok, A. B. Abdullah, N. Z. Jhanjhi, and M. Supramaniam, "Ransomware: Threat and detection techniques—A review," *Int. J. Comput. Sci. Netw. Secur.*, vol. 19, no. 10, pp. 136–146, Oct. 2019.

[8] K. Higuchi and R. Kobayashi, "Real-time open-file backup system with machine-learning detection model for ransomware," *Int. J. Inf. Secur.*, vol. 24, no. 1, p. 54, Jan. 2025.

[9] K. Gujar, P. Jagdale, S. Yadav, and S. Bhattacharjee, "Backup solid state drive for ransomware protection," *J. Operating Syst. Develop. Trends*, vol. 10, no. 1, pp. 12–18, 2023.

[10] B. Reidys, P. Liu, and J. Huang, "RSSD: Defend against ransomware with hardware-isolated network-storage codesign and post-attack analysis," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Feb. 2022, pp. 726–739.

[11] S. Baek, Y. Jung, D. Mohaisen, S. Lee, and D. Nyang, "SSD-assisted ransomware detection and data recovery techniques," *IEEE Trans. Comput.*, vol. 70, no. 10, pp. 1762–1776, Oct. 2021.

[12] W. H. Mir, N. Goel, and V. K. Tavva, "CARDR: DRAM cache assisted ransomware detection and recovery in SSDs," in *Proc. Int. Symp. Memory Syst.*, Sep. 2024, pp. 104–115.

[13] H. Fujinoki and L. Manukonda, "Proactive damage prevention from zero-day ransomwares," in *Proc. 5th Int. Conf. Comput. Commun. Internet (ICCCI)*, Jun. 2023, pp. 133–141.

[14] W. Zhu, C. Stillman, S. Rampazzi, and K. R. B. Butler, "Enabling secure and efficient data loss prevention with a retention-aware versioning SSD," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 171–185, doi: 10.1145/3719027.3765135.
[15] Z. Wei-dong, G. Hernandez, W. García, D. Tian, S. Rampazzi, and K. Butler, "SrFTL: Leveraging storage semantics for effective ransomware defense in flash-based SSDs," *ACM Trans. Storage*, vol. 21, no. 4, pp. 1–42, 2025, doi: 10.1145/3767322.
[16] D. Min, Y. Ko, R. Walker, J. Lee, and Y. Kim, "A content-based ransomware detection and backup solid-state drive for ransomware defense," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 7, pp. 2038–2051, Jul. 2022.
[17] K. Friday and E. Bou-Harb, "Accelerating ransomware defenses with computational storage drive-based API call sequence classification," in *Proc. 17th Cyber Secur. Experimentation Test Workshop*, Aug. 2024, pp. 8–16, doi: 10.1145/3675741.3675743.
[18] J. A. Gómez-Hernández, L. Álvarez-González, and P. García-Teodoro, "R-locker: Thwarting ransomware action through a honeyfile-based approach," *Comput. Secur.*, vol. 73, pp. 389–398, Mar. 2018.
[19] D. Zhuravchak, T. Ustyianovych, V. Dudykevych, B. Venny, and K. Ruda, "Ransomware prevention system design based on file symbolic linking honeypots," in *Proc. 11th IEEE Int. Conf. Intell. Data Acquisition Adv. Comput. Systems: Technol. Appl. (IDAACS)*, vol. 1, Sep. 2021, pp. 284–287.
[20] S. Lee, H. K. Kim, and K. Kim, "Ransomware protection using the moving target defense perspective," *Comput. Electr. Eng.*, vol. 78, pp. 288–299, Sep. 2019.
[21] S. Kok, A. Abdullah, N. Jhanjhi, and M. Supramaniam, "Prevention of crypto-ransomware using a pre-encryption detection algorithm," *Computers*, vol. 8, no. 4, p. 79, Nov. 2019.
[22] S. Song, B. Kim, and S. Lee, "The effective ransomware prevention technique using process monitoring on Android platform," *Mobile Inf. Syst.*, vol. 2016, pp. 1–9, May 2016.
[23] G. Cusack, O. Michel, and E. Keller, "Machine learning-based detection of ransomware using SDN," in *Proc. ACM Int. Workshop Secur. Softw. Defined Netw. Netw. Function Virtualization*, Mar. 2018, pp. 1–6.
[24] V. Oujezsky, P. Novak, T. Horvath, M. Holik, and M. Jurcik, "Data backup system with integrated active protection against ransomware," in *Proc. 46th Int. Conf. Telecommun. Signal Process. (TSP)*, Jul. 2023, pp. 65–69.
[25] M. E. Ahmed, H. Kim, S. Camtepe, and S. Nepal, "Peeler: Profiling kernel-level events to detect ransomware," in *Proc. Comput. Secur.–ESORICS*, 2021, pp. 240–260.
[26] G. O. Ganfure, C.-F. Wu, Y.-H. Chang, and W.-K. Shih, "RTrap: Trapping and containing ransomware with machine learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1433–1448, 2023.
[27] P. M. Anand, P. V. S. Charan, H. Chunduri, and S. K. Shukla, "LARM: Linux anti ransomware monitor," *Comput. Secur.*, vol. 159, Dec. 2025, Art. no. 104700.
[28] S. Wang, F. Dong, H. Yang, J. Xu, and H. Wang, "Cancal: Towards real-time and lightweight ransomware detection and response in industrial environments," in *Proc. CCS*, Nov. 2024, pp. 2326–2340, doi: 10.1145/3658644.3690269.
[29] J. V. D. Assen, C. Feng, A. H. Celdrán, R. Oleš, G. Bovet, and B. Stiller, "Guardfs: A file system for integrated detection and mitigation of linux-based ransomware," *J. Inf. Secur. Appl.*, Sep. 2025, Art. no. 104078. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214212625001152
[30] (2023). *What is Ebpf? An Introduction and Deep Dive Into the Ebpf Technology*. [Online]. Available: https://ebpf.io/what-is-ebpf
[31] (2023). *BCC-Tools for Bpf-Based Linux Io Analysis, Networking, Monitoring, and More*. [Online]. Available: https://github.com/iovisor/bcc
[32] K. Higuchi and R. Kobayashi, "ROFBSα: Real time backup system decoupled from ML based ransomware detection," 2025, *arXiv:2504.14162*.
[33] S. Devkar. (Feb. 2022). *Avoslocker-Modern Linux Ransomware Threats*. Accessed: Jan. 24, 2026. [Online]. Available: https://blogs.vmware.com/security/2022/02/avoslocker-modern-linux-ransomware-threats.html
[34] Y. Lemmou, J. Lanet, and E. M. Souidi, "In-depth analysis of ransom note files," *Comput.*, vol. 10, no. 11, p. 145, 2021. [Online]. Available: https://www.mdpi.com/2073-431X/10/11/145
[35] J. Jia, Y. Zhu, D. Williams, A. Arcangeli, C. Canella, H. Franke, T. Feldman-Fitzthum, D. Skarlatos, D. Gruss, and T. Xu, "Programmable system call security with ebpf," 2023, *arXiv:2302.10366*.
[36] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacífico, E. R. S. Santos, E. P. M. C. Júnior, and L. F. M. Vieira, "Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications," *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–36, Feb. 2020, doi: 10.1145/3371038.
[37] The Linux Kernel documentation. (2026). *Control Group V2*. [Online]. Available: https://docs.kernel.org/admin-guide/cgroup-v2.html
[38] (2026). *Cfs Scheduler*. [Online]. Available: https://docs.kernel.org/scheduler/sched-design-CFS.html
[39] (2026). *Extensible Scheduler Class*. [Online]. Available: https://docs.kernel.org/scheduler/sched-ext.html
[40] (2026). *Dm-Crypt*. [Online]. Available: https://docs.kernel.org/admin-guide/device-mapper/dm-crypt.html
[41] The GNU Privacy Handbook. (2026). *Encrypting and Decrypting Documents*. [Online]. Available: https://www.gnupg.org/gph/en/manual/x110.html
[42] VMware Security Blog. *Avoslocker–Modern Linux Ransomware Threats*. Accessed: Apr. 24, 2026. [Online]. Available: https://blogs.vmware.com/security/2022/02/avoslocker-modern-linux-ransomware-threats.html
[43] Unit 42. (2021). *Ransomware Groups To Watch: Emerging Threats*. [Online]. Available: https://unit42.paloaltonetworks.com/emerging-ransomware-groups/
[44] VMware Security Blog. (2022). *Esxi-Targeting Ransomware: The Threats That Are After Your Virtual Machines (part 1)*. [Online]. Available: https://blogs.vmware.com/security/2022/09/esxi-targeting-ransomware-the-threats-that-are-after-your-virtual-machines-part-1.html



KOSUKE HIGUCHI received the B.E. degree in information science from Chiba Institute of Technology, in 2023, and the M.E. degree in information and communications from Kogakuin University, in 2025, where he is currently pursuing the Ph.D. degree. His research interests include modeling and mitigating ransomware encryption attacks.



RYOTARO KOBAYASHI (Member, IEEE) received the B.E., M.E., and D.E. degrees from Nagoya University, in 1995, 1997, and 2001, respectively. He was a Research Associate with Nagoya University, from 2000 to 2008. From 2008 to 2017, he was with Toyohashi University of Technology, where he was a Lecturer and later as an Associate Professor. He joined Kogakuin University as an Associate Professor, in 2017, and has been a Professor, since 2019. His research interests include system security, malware detection using machine learning, hardware-assisted security, and system forensics. He is a member of the Information Processing Society of Japan (IPSJ).